

Never Twice the Same Mistake: Counterexample-Guided Program Repair with a Typed Memory of Refuted Attempts

Anonymous Author(s)

ABSTRACT

Large language model repair agents iterate: propose a patch, run the tests, read the failure, try again. Recent measurement shows how expensive the running is and how little of it pays—agents average 8.8 test executions per SWE-bench task and “apply execution indiscriminately.” Classical repair cuts that work either by relating candidate patches to one another, which an unenumerable candidate space rules out, or by ordering the tests so a doomed candidate dies on the first, which nothing rules out.

We carry the second mechanism into the LLM loop and change what it orders from. CEGMEM retains counterexamples and indexes them by edit location—the half of a failure label a diff supplies without executing anything—then re-executes a stored one against each new candidate, resolving the round as soon as an input already in hand still breaks it. In the synthesis loop this idea comes from such replay would be vacuous, since a constrained synthesizer cannot emit a candidate its own counterexamples refute; an LLM repeats itself in one proposal out of four. We prove guard soundness against a finite oracle, a $K+1$ round bound and strict redundancy reduction, and show the index is *not* asymptotically cheaper than a flat scan.

We then run the loop on 99 ConDefects faults under two proposers, a local `gwen2.5-coder:7b` and `gpt-4o-mini`, five arms and 1,485 pre-specified main-grid cells per proposer, every arm paired by common random numbers so round 1 is the identical draw in all of them. The guard is proposer-independent: alone, it cuts oracle calls per episode $\times 5.0$ and $\times 5.8$ —counting guard replays as the executions they are, total program executions $\times 2.2$ and $\times 2.1$ —at unchanged repair rate, and pays for a redundant verification $\times 44$ and $\times 46$ less often. At a matched budget of two oracle calls it lifts repair from 31.9% to 55.8% (task-clustered 95% CI on the gain [18.4, 29.5] pp). Replayed against classical test prioritizers on identical candidates the execution saving reverses; what survives, on every task, is the oracle-call reduction no prioritizer moves. Prompt-side steering is proposer-dependent, and a pre-specified falsifier records which way: null on every outcome at 7B for +128% prompt tokens, and +9 pp repair for +67% tokens on `gpt-4o-mini`. We recommend the guard unconditionally and steering only where the proposer acts on it.

1 INTRODUCTION

Automated program repair has moved from search- and template-based systems to agents built on large language models that repair real projects by interacting with an environment:

localize a fault, propose an edit, run the test suite, read the failure, revise [2, 24, 25, 28, 29]. That loop is counterexample-guided repair by another name—each failing test is a counterexample that ought to constrain the next proposal, exactly as a failing input drives the synthesis loop of SKETCH [18].

The cost of that loop is now measured. Analyzing 7,745 SWE-bench agent traces and 3,000 end-to-end repair runs, Lin et al. [10] report that agents average 8.8 test executions per task, that the benefit concentrates on a minority of instances, and that restricting execution entirely costs only 1.25 percentage points of resolve rate; their conclusion is that “current agents apply execution indiscriminately, paying their cost on instances where it provides little benefit.” Agent-authored tests likewise “mimic a familiar software-development practice while consuming interaction budget” [5], and counting iteration honestly makes self-repair’s gains “often modest” [14].

Three responses exist. Systems work makes each validation cheaper [3, 26]; scaling work buys accuracy with money, most explicitly CodeMonkeys at $\approx \$2,300$ for 57.4% of SWE-bench Verified [6]; and classical generate-and-validate APR cuts the work itself, either by relating candidates to each other [12, 22], which needs an enumerable candidate space an LLM does not supply, or by ordering the tests so a doomed candidate dies on the first [11, 16, 20], which needs nothing of the candidate space at all. The second mechanism transfers. Our question is what it should order from when the generator is a language model.

A refutation you already hold is such a criterion. If memory contains a counterexample x that a candidate p still fails, p is refuted and validating it further buys nothing. In the synthesis loop this idea comes from, the criterion could never fire—a constrained synthesizer cannot emit a candidate that its accumulated counterexamples refute—but an LLM proposer is under no such obligation and repeats itself in one proposal out of four (Section 2.3). The obstacle is that today’s agents keep a flat conversational transcript [4, 17, 24], which neither *retrieves*—it dilutes as the dialogue grows—nor *generalizes*: it records that patch p_3 failed test t , not that every patch sharing p_3 ’s defect will fail the same way.

Our approach: keep the evidence, index it by where the patch edits. When a proposal p is refuted by counterexample x , CEGMEM stores (p, x, τ) where $\tau = \theta(p, x)$ pairs p ’s edit location with the property the execution violated. The halves are available at different times, and the mechanism rests on that: $\lambda(p)$ is a diff against the buggy source and needs no execution,

while the violated property exists only after a failing run. So the store is *indexed* by λ and can be consulted for a candidate nobody has run.

That buys two operations. The agent can *guard*: re-execute a stored counterexample against the new candidate and, if it still fails, resolve the round without calling the oracle—a decision that is verified rather than predicted, since the candidate is blocked only after failing an input the oracle itself produced. Inlined into the oracle this is a prioritization discipline that stops on the first failing case, and we claim no more than that; what is new is that the order comes from counterexamples the loop discovered itself and kept across candidates. And it can *steer*: remove already-refuted classes from the proposer’s support. Our study finds the first carries its value on every proposer we ran and the second only on one of them, which is not what we set out to show.

This paper. We give CEGMEM a formal core (Section 4) and run it on real faults rather than the simulation the design was first argued from: 99 ConDefects faults, a local `qwen2.5-coder:7b` proposer at $T = 1.0$, a differential oracle over the shipped pool, five arms and four sweeps, 1,485 pre-specified main-grid cells and 3,654 episodes, and the main arms repeated under `gpt-4o-mini` (1,782 further episodes). Two commitments make the comparison tight: every arm is paired by *common random numbers* [9], so round 1 is the byte-identical draw in every arm and later divergence is the treatment’s own doing; and the primary metric is pre-specified as repair rate at a matched *oracle-call* budget, not total calls, which an arm without a guard loses by construction.

Memory cuts oracle calls $\times 5.2$ and, counting every guard replay as the execution it is, total program executions $\times 2.2$, at unchanged repair rate; at a matched two-call budget it lifts repair from 31.9% to 55.8%. Against a classical prioritizer replayed on identical candidates the execution saving reverses and only the oracle-call reduction survives (Section 7.6). We then repeat the main arms under a second proposer, `gpt-4o-mini`, against two criteria fixed in advance (Section 7.7). The guard’s reduction transfers intact— $\times 5.8$ fewer oracle calls and $\times 46$ fewer paid redundant verifications on its own, $\times 93$ in the full typed agent, repair unchanged. The prompt-side half does not behave the same way: at 7B, steering is null on every outcome metric while costing +128% prompt tokens and making the full agent the slowest of five arms; on `gpt-4o-mini` it lifts repair by +9 pp for +67% tokens. The pre-specified failure rule for that criterion fires, and the recommendation is consequently the guard unconditionally, with steering only where the proposer acts on it.

Contributions. (i) An architecture separating verification-side guarding from prompt-side steering into independently analyzable operators (Section 3). (ii) A formal core stated against a *finite* oracle: guard soundness, a $K+1$ round bound, strict redundancy reduction, and an explicit account of what indexing does and does not buy (Section 4). (iii) A controlled execution of that loop on real faults under common random numbers, with pre-specified metrics, falsifiers and a random-partition

control, replicated under a second proposer (Sections 6 and 7). (iv) A separation of the two halves by what they depend on: the guard’s saving is a property of the runtime and transfers across proposers; steering’s value is a property of the proposer, null at 7B and positive on `gpt-4o-mini`, which is what relocates the unconditional recommendation onto the guard (Sections 7.7 and 8).

2 BACKGROUND AND RELATED WORK

Deciding that one candidate patch needs less validation work than the next is not new, and Table 1 sorts the ways it has been done by what the decision is computed from and what that demands of the candidate space. We put CEGMEM in the same row as a mechanism a decade older than LLM repair, and then say what is left over.

2.1 Removing validations: candidate equivalence

One family relates candidates *to each other* and validates one representative per class. AE quotients the patch space under an approximate program-equivalence relation: on the 45 defects it and GenProg both repair, from a 105-defect benchmark, it needs 186 test-suite evaluations against 3,252 [22]. Test-equivalence analysis partitions candidates by the values a modified expression evaluates to during a test, so one execution decides an entire class [12], and ExpressAPR detects that equivalence *online*, reporting $\approx 137\times$ over plain validation on Defects4J [26]. UniAPR, by contrast, purely accelerates—it redefines bytecode inside a warm JVM and still runs every test [3]. The equivalence mechanisms give a stronger guarantee than ours, and they need what an LLM does not supply: an enumerable candidate space and an edit shape the analysis can model. That is why they do not transfer, and it is the only reason.

2.2 Reordering validations, and where our guard belongs

The other family validates every candidate and changes the order in which evidence against it is sought. Fault-recorded prioritization runs the tests that killed earlier candidates first [16]; modification-point-aware prioritization keeps one ordered suite per edit location [20]; AE’s TestStrat—in the same paper as its quotient space—takes the test likeliest to fail and stops there [22]; and a study of 2,532,915 patches from 12 repair systems on 395 Defects4J bugs quantifies what selection and prioritization save [11].

Our guard belongs here, and we state the identity rather than argue with it. Inline the guard into the oracle—stored counterexamples first, stop at the first failure—and the executions performed are the same ones, in the same order, at the same cost. Nothing is *removed*; a validation is made to terminate on its first case. We therefore claim an ordering effect and report it as one: in TYPED a round the guard resolves costs 1.13 program executions where a round reaching the oracle costs 18.9, and memory turns roughly 7.6 oracle calls per episode into roughly 7.4 early-terminating ones (Section 7.1). What is *not* shared with prior work is the key and the evidence. Those tables rank the project’s own tests by a failure history over candidates; ours holds inputs discovered inside the episode by candidates the oracle already refuted, re-executed against a

Table 1: Where CEGMEM sits. The columns are factual properties of each mechanism, not a scoring of it. CEGMEM shares the last column’s value with a line a decade older than LLM repair; what is new is not a cheaper validation—a classical prioritizer beats ours on executions (Section 7.6)—but that the same evidence removes oracle invocations, which no row above does, from a key that needs nothing of the candidate space.

Line of work	Representatives	The decision is computed from	Requires of the candidate space	Effect on validation work
Candidate equivalence	AE [22]; f1x [12]	candidates related to <i>each other</i> : syntactically, or by the values a modified expression takes	enumerable; an edit shape the analysis models	removes
Validation engineering	UniAPR [3]; ExpressAPR [26]	a warm JVM; online patch equivalence, schemata, virtualization	mutation-shaped patches, for the equivalence part	accelerates; ExpressAPR also removes
Test prioritization	fault-recorded [16]; modification-point aware [20]; AE’s TestStrat [22]; RTS study [11]	which of the project’s own tests killed earlier candidates, keyed on nothing or on the edit location	nothing	reorders, stops early
LLM repair agents	RepairAgent [2]; SWE-agent [28]; Agentless [25]; ChatRepair [24]; ExpeRepair [13]; [21]	a flat transcript, or demonstrations and insights, retrieved into the prompt	nothing	none
Counterexample-guided repair	SKETCH [18]; LLM CEGIS [15]	counterexamples, as prompt content for the next attempt	a complete synthesizer (SKETCH); nothing (LLM)	none
CEGMEM	this paper	counterexamples <i>retained across candidates</i> and re-executed, indexed by edit location λ	nothing	reorders, stops early; <i>removes oracle invocations</i>

candidate nobody has run, indexed by a location a diff supplies. The per-modification-point table of [20] is the nearest neighbour, and we run it: over byte-identical candidates it reaches a refuting case in fewer executions than our guard on 80 of 99 tasks, by a median 4.6% (Section 7.6). What separates the two is not the cost of a validation but whether one happens: no prioritizer in this line moves oracle invocations.

2.3 Counterexample-guided repair, and why replay is not vacuous

That a counterexample should constrain the next candidate is the engine of the loop SKETCH introduced [18], and the loop has been run with an LLM in the synthesizer’s place over 1,431 student C programs [15]. There, retaining counterexamples and replaying them would be *vacuous*: a constrained synthesizer cannot emit a candidate one of them refutes, so a guard would never fire. Losing that constraint is what creates the opportunity, and an LLM takes it often. In GUARD-ONLY, 24.2% of proposals are byte-identical to an earlier one in the same episode and 30.2% of the rounds the guard resolves are such a repeat; of those resolutions 12.8% re-propose a patch the store already holds, and the remaining 87.2% are patches the store never held, refuted by a stored input anyway (Table 2). Neither case can arise in SKETCH. Orvalho et al. use the counterexample as prompt content for the next attempt—our STEER-ONLY arm, the half we find null; we add the second use of the same evidence.

2.4 Memory in LLM repair agents

LLMs repair code by conversing: ChatRepair iterates on test failures [24], agentic systems localize, edit and validate over many turns [2, 28, 29], “agentless” pipelines keep a fixed propose–validate workflow [25], and general frameworks add verbal self-reflection [4, 17]. In all of them the record of past attempts is a flat transcript and the cost model is dollars or tokens. The closest memory-augmented system, ExpeRepair [13], retrieves demonstrations and reflective insights to condition generation but never validation; nothing in that index recognizes that a new candidate is refuted by an input already

Table 2: What makes replaying a stored counterexample non-vacuous, and what the λ index does not buy. Every quantity is derived by replaying the frozen round log—no program was executed to produce this table (tools/related_addenda.py). The upper block is the opportunity an LLM proposer creates and a constrained synthesizer does not; the lower block is why we claim no benefit for indexing on this corpus. The last two rows of the upper block partition the resolutions; the repeat-proposal row above them cuts across that partition.

	UNTYPED	TYPED	GUARD-ONLY
<i>The proposer repeats itself</i>			
Rounds the guard resolved	3785	3649	2253
Proposals made by the arm	4732	4557	2807
byte-identical to an earlier one	24.3%	27.1%	24.2%
Of resolutions, a repeat proposal	30.4%	33.9%	30.2%
a patch the store already held	12.6%	13.8%	12.8%
a patch the store never held	87.4%	86.2%	87.2%
<i>Memory is too small for order to matter</i>			
Entries held at resolution, median	1	1	1
maximum observed	5	4	3
Resolutions with one entry or none	57.5%	63.3%	60.5%
Resolutions after one consultation	83.3%	87.6%	86.4%
<i>Some evidence is found, not given</i>			
Distinct stored inputs per episode	1.53	1.44	1.47
Stored inputs beyond the round-1 one	34.7%	30.5%	32.0%

in hand. Vo et al. [21] type failures for text-to-SQL repair, conditioning *retrieval for the prompt* where we condition *the order of validation*, and report their typed variant “is not reliably separated from untyped dynamic retrieval”—the phenomenon we measure directly. The guard is a property of memory, not of a controller, so it is orthogonal to every agent above. Finally, since a patch can pass the visible tests and still be wrong [27] and LLM-era benchmarks must worry about leakage, we use ConDefects [23] rather than Defects4J [8] or SWE-bench [7], and treat acceptance as a claim about G_k audited against G_{pool} (Section 7.4).

Table 3: Notation. The three horizontal blocks are the distinctions this paper depends on and that its earlier drafts blurred: what is correct versus what an oracle certifies; what is computable before an execution versus after one; and which budget a number is denominated in.

Symbol	Meaning
$\mathcal{P}, p$	candidate space; a candidate patch
$\phi, \text{sat}(p)$	specification; p satisfies ϕ on all of $\mathcal{X}$
$\mathcal{G}$	stochastic proposer (the LLM)
π, q_τ	base mass on the correct class; on type τ
w_s	base mass of class s ($w_\top = \pi, w_\tau = q_\tau$)
$G^* \subseteq \mathcal{P}$	semantically correct patches
G_{pool}	patches passing the entire shipped test pool
G_k	patches the depth- k oracle accepts; $G^* \subseteq G_{\text{pool}} \subseteq G_k$
$\mathcal{O}_k(p)$	counterexample oracle at depth k (Eq. (1))
$\lambda(p)$	edit location: the key computable <i>before</i> execution
$\theta(p, x) \in \mathcal{T}$	failure label, defined <i>after</i> x refutes p
$\mathcal{M}, \mathcal{M}[\ell]$	memory of triples (p, x, τ) ; its $\lambda = \ell$ bucket
$E \subseteq \mathcal{T}$	labels this arm has eliminated
$E^{\text{obs}} \subseteq \mathcal{T}$	labels refuted so far, arm-neutral
K	reachable failure types, $ \{\tau : q_\tau > 0\} $
ρ, c	oracle informativeness; typing coherence (Definition 1)
B	proposal budget per episode (20 throughout)
b	oracle-call budget: the reported cost axis
k	oracle depth: test cases per verification

3 THE CEGMEM ARCHITECTURE

3.1 Problem setup

A repair task is a tuple $(\mathcal{P}, \phi, \mathcal{O}_k)$: a candidate space of patches, a specification, and an oracle. A patch is *semantically correct* iff it satisfies ϕ on every input in a domain $\mathcal{X}$, and we write G^* for that set, assumed non-empty. No implementable oracle decides membership in G^* . What we have is a *counterexample oracle at depth k* : a call draws a check set X_k of $\min(k, |\text{pool}|)$ inputs from the fault’s shipped pool and runs them until one separates candidate from reference,

$$\mathcal{O}_k(p) = \begin{cases} x \in X_k \text{ with } \neg\phi(p, x) & \text{if a checked input fails,} \\ \text{accept} & \text{otherwise.} \end{cases} \quad (1)$$

Write G_k for what one such call certifies—the patches passing every $x \in X_k$ —and G_{pool} for those passing the *entire* pool. Two properties of X_k are load-bearing. It is drawn *per call*, so G_k is a family indexed by the draw; but the draw is degenerate whenever $k \geq |\text{pool}|$, where X_k is the pool and $G_k = G_{\text{pool}}$ —the case for 97 of our 99 faults at the $k = 100$ every headline number uses. The three sets nest, $G^* \subseteq G_{\text{pool}} \subseteq G_k$, so acceptance is the weakest of them and $G_k \setminus G^*$ is *false acceptance*; Section 7.4 measures that gap and never assumes it away. Three budget quantities also stay distinct: the proposal budget B , the oracle budget b against which repair rate is reported, and the oracle depth k .

3.2 Failure labels, and the key available before execution

A refuted candidate p with counterexample x receives a label

$$\theta(p, x) = (\lambda(p), \text{property violated at } x) \in \mathcal{T}, \quad (2)$$

where $\lambda(p)$ is the patch’s *edit location*, obtained by diffing the candidate against the buggy source. The two components differ in when they are available, and that is load-bearing: $\lambda(p)$ needs no execution, while the violated property is only defined *after* a failing run and so does not exist for a fresh candidate. The guard therefore cannot index on θ . It indexes on λ , and memory’s buckets are keyed on it: $\mathcal{M}[\ell] = \{(p', x', \tau') \in \mathcal{M} : \lambda(p') = \ell\}$. The full label θ is what steering excludes—it names classes already *observed*, for which the property component exists—and what the coherence sweep perturbs. Nothing here requires predicting a violated property before running anything.

Definition 1 (Informativeness and coherence). The oracle is ρ -*informative* if a counterexample refuting a candidate of class τ refutes every candidate of class τ with probability at least ρ . Memory is c -*coherent* if it attributes a candidate or counterexample to its true class with probability at least c . The ideal regime is $\rho = c = 1$.

3.3 Typed memory: guard and steer

CEGMEM maintains a memory $\mathcal{M}$ of triples (p, x, τ) and the induced set of eliminated classes E . It exposes two operations, kept distinct because Section 7 shows they do different jobs and that only one works.

Guard. Before paying the oracle, the guard rejects a candidate that a stored counterexample still refutes:

$$\text{Guard}_{\mathcal{M}}(p) = \neg \exists (p', x', \tau') \in \mathcal{M} : x' \text{ refutes } p]. \quad (3)$$

Here x' *refutes* p under the oracle’s own predicate— p on x' disagrees with the reference, crashes, or times out—so a block and a refutation are the same event, evaluated by different callers. An *indexed* guard evaluates Eq. (3) against $\mathcal{M}[\lambda(p)]$ first and then the rest of memory—the fallback belongs to the analyzed algorithm and to the shipped one, not to one of them—while an *untyped* guard walks memory in insertion order. Both short-circuit at the first entry that still refutes and both consult all of memory when none does, so their worst cases coincide: the index changes the *order* of consultation, not its bound (Proposition 7). Our implementation additionally deduplicates stored counterexamples by input and memoizes verdicts within a round, reducing executions at unchanged consultations.

Two properties of Eq. (3) are easy to lose. It is a *verified* predicate, not a match heuristic: a block happens only when a stored counterexample is re-executed and still fails, so widening the search can never cost soundness (Theorem 2). And the index must say where to look *first*, not where to *stop*: λ records where a candidate edits while refutation is decided by which input it gets wrong, so a bucket-only guard is an unsound stopping rule. Section 5 reports what that cost us.

Steer. Memory also reshapes the proposer. Let w_s be its base mass on class s , with $w_\top = \pi$ for the correct class and $w_\tau = q_\tau$ otherwise. The steered proposer renormalizes onto

Algorithm 1: CEGMEM: counterexample-guided repair with typed memory. The two memory operations are lines 3 (steer) and 4 (guard); switching each off independently yields the five arms of Section 6.

Input : task $(\mathcal{P}, \phi, \mathcal{O}_k)$, proposer $\mathcal{G}$, budget B
Output : a repaired patch, or $\perp$

```

1  $\mathcal{M} \leftarrow \emptyset; E \leftarrow \emptyset$  // typed memory;
   eliminated types
2 for  $t \leftarrow 1$  to  $B$  do
3    $p_t \sim \mathcal{G}(\cdot | E)$  // steer: avoid
   eliminated types, Eq. (4)
4   if  $\text{Guard}_{\mathcal{M}}(p_t)$  then
5     continue // a stored
   counterexample already refutes
    $p_t$ , Eq. (3)
6    $r \leftarrow \mathcal{O}_k(p_t)$  // the expensive
   verification round
7   if  $r = \text{accept}$  then
8     return  $p_t$  //  $p_t$  passed every  $x \in X_k$ ,
   Theorem 2
9    $x_t \leftarrow r; \tau_t \leftarrow \theta(p_t, x_t)$ 
10   $\mathcal{M} \leftarrow \mathcal{M} \cup \{(p_t, x_t, \tau_t)\}; E \leftarrow E \cup \{\tau_t\}$ 
   // store + eliminate the class
11 return  $\perp$ 

```

the surviving support:

$$\mathbb{P}_{\text{steer}}[s] = \frac{w_s \cdot \mathbb{K}[s \notin E]}{\pi + \sum_{\tau \notin E} q\tau}, \quad s \in \{\top\} \cup \mathcal{T}. \quad (4)$$

This is what a flat transcript cannot do—excluding a class requires having one—and it idealizes an operation an LLM can only be *asked* to perform: an exclusion instruction plus class-matched evidence in the prompt. Whether the request is honored is empirical; Section 7.3 answers no for a 7B proposer, while the same renormalization applied on the guard side, by not charging blocked rounds, does deliver (Section 7.3).

3.4 The repair loop

Each round draws a steered proposal, guards it against memory, and calls the oracle only if it survives; an accepted patch is returned and a refuted one stored with its counterexample and label, eliminating that class. The arms in Section 7 switch the two operations independently (Algorithm 1 gives the loop in full). CEGMEM adds no planner and no controller—the only new state is the memory—which is what keeps Section 4’s guarantees properties of *memory*, and so independent of the proposer.

4 FORMAL CORE

Two conventions govern this section. Every guarantee is about what a finite oracle certifies— G_{pool} where we can get it, G_k where the depth matters—and never silently about G^* ; and the guard is analyzed as an *index*, which changes the order in which stored counterexamples are consulted but not the

worst case, so we claim a measured cost reduction and not an asymptotic separation. Statements and proofs of everything below are in Appendix A.

Assumption 1 (Refutation soundness). When $\mathcal{O}_k$ refutes a candidate it returns a genuine witness: if $\mathcal{O}_k(p) = x$ then $\neg\phi(p, x)$.

Assumption 1 constrains *refutation*, not acceptance. It is the only oracle property the guard needs; it holds for any oracle reporting a failing execution it actually ran, and for ours it also requires the reference to be right, which the corpus gate enforces (Section 6). It has one soft edge: a *timeout* verdict is a resource fact rather than a witness of $\neg\phi$, and the one fault whose candidates sit at the sandbox limit produces every cross-arm inconsistency we observe (Section 7.8). The converse—that acceptance implies correctness—is assumed nowhere: at depth k it is false in general, and Section 7.4 measures how false.

Theorem 2 (Guard soundness). *Under Assumption 1, if $\text{Guard}_{\mathcal{M}}(p) = 1$ with witness x then $p \notin G_{\text{pool}}$, and hence $p \notin G^*$; and $p \notin G_k$ for every check set containing x , which includes every X_k that exhausts the pool. No candidate such a call would have accepted is ever blocked, however widely the guard searches. If CEGMEM returns $\hat{p}$, then $\hat{p}$ passed every input its accepting call drew.*

The theorem is about the guard, not the oracle: Eq. (3) blocks only on a counterexample that is *re-executed and still fails*, so widening the search can only find more true refutations, and nothing here says $\hat{p} \in G^*$. Its second clause can fail only on a redrawn X_k that omits the witness, so we test rather than assume: an audit condition pays the oracle on blocked rounds and asks whether it accepts, and over 4,411 blocked rounds it never did. That is not a tight bound on the hazard—where X_k exhausts the pool the audit confirms what already holds by construction, and below $k = 100$, where the hazard is real, it was not run—but it does establish that the implemented guard blocks nothing the implemented oracle accepts. This is falsifier F1, with the requirement that a guarded arm’s repair rate equal its unguarded twin’s.

Corollary 3 (Budgeted success). *For any guard sound in the sense of Theorem 2: if blocked rounds are not charged against the proposal budget B , the probability of repairing within B charged proposals cannot fall, and rises strictly only on realizations where a block moves the first accepting draw from beyond B to within it. If blocked rounds are charged, the guard is outcome-neutral: under common random numbers the round of first acceptance is invariant to it.*

Corollary 3 shaped the study and applies to untyped memory as much as typed. Under the accounting Section 7 uses—one proposal, one budget unit—the guard cannot change which round first accepts, so any gap between a guarded arm and its unguarded twin is a soundness bug, not a result (Section 7.8); it also makes the first half untestable there, hence the free-guarded condition (Section 7.3).

Three further results, stated in full in the supplement. Under an i.i.d. proposer and *ideal typing*—which also requires refutation to be class-exclusive, an idealization our λ index does not meet—typed memory halts on an unbounded run within $K+1$ oracle calls and $K+1$ proposals, K being the reachable failure classes, while untyped memory attains the call bound and no proposal bound. Expected oracle calls fall from $1/\pi$ to $1+D$ with either memory, $D = \sum_{\tau} q_{\tau}/(q_{\tau} + \pi)$. And counting redundancy against the *observed-type history*—the classes already refuted this episode whether or not the arm records them, which is what makes it measurable in a memory-free arm—redundant attempts *paid* fall to 0 with either memory, while those *generated* fall to 0 only with typed memory, which needs steering to work; Section 7.3 reports that it does not. Of the three budget quantities only two are bounded: not *charged* proposals, which include blocked rounds. These are directional predictions, and Section 7.5 reports which of the closed forms the data supports.

What the index costs. Our guard consults the λ -matched bucket first and then the rest of memory (Section 5). Both it and a flat scan short-circuit at the first entry that still refutes, and both consult all of memory when none does, so their worst cases coincide and no asymptotic separation exists. What differs is where in the order a refuting entry is found: the index saves to the extent that refuting entries concentrate in the bucket—35% of guard decisions in vivo, a rate the random-partition control matches (Section 7.2)—and pays $|\mathcal{M}[\lambda(p)]|$ extra consultations on a miss. On this corpus there is little order to exploit: memory holds a median of one entry when the guard fires, and 86% of blocks resolve on the first consultation. Separately, deduplicating stored counterexamples by input and memoizing verdicts within a round reduce *executions* at unchanged *consultations*—which is the signature Section 7.3 finds, and the reason our claim there is a measured constant factor.

5 IMPLEMENTATION

CEGMEM is 4,700 lines of Python. Three components matter, plus one design error whose correction is the difference between a null result and the paper’s main finding.

Proposer and oracle. One LLM call per round returns a whole rewritten program in a fenced block. The prompt carries the buggy source, worked input/output examples (identical in every arm, so the specification cannot confound the comparison) and—only in the steered arm—the evidence and exclusion blocks of Eq. (4). Output budget is sized per program so a truncated reply cannot enter memory as a *harness* failure; truncations, overflows and backend errors are logged as spent rounds that leave memory untouched. The oracle $\mathcal{O}_k$ is a differential tester: candidate and reference run on a per-call sample X_k of the fault’s shipped pool in a sandbox with a 30 s per-case limit, and the first disagreement is returned as a case *name*, so re-executing it later is cheap. Acceptance means agreement on every case in X_k ; the reference is never visible to the proposer.

The type map, and the key the guard can use. θ is the pair (edit location, violated property). The location is a pure diff against the buggy source and so is computable *before* execution, which is what makes it usable as the guard’s index λ ; the property is read off the failing execution—wrong output, uncaught exception, or timeout. We report the *fine* granularity (line-range location); a *coarse* variant collapsing it to the whole program measures how much the location half contributes (Section 7.2).

An implementation correction. Our first guard searched the λ -matched bucket *and nothing else*, which makes the index a *partition*—sound as a stopping rule only if its key decides refutation. It does not: the key is where a candidate edits, refutation is decided by which input it gets wrong, and the two are close to independent. The frozen log prices the mistake exactly: of the typed rounds that reached the oracle and were refuted, 68.9% came back carrying a counterexample *already in memory*—every one a round the flat scan would have blocked. The fix is fifteen lines: search the bucket first, then fall back to the rest of memory, deduplicating by counterexample input and memoizing verdicts within the call—which is also why this paper claims no asymptotic advantage for the index, since the two guards then share a worst case. Every number here is from the corrected guard; we report the error because a bucket-only guard is the natural first implementation and it silently turns this paper’s result into a null one.

Instrumentation and replay. One record per round carries the patch, verdict, counterexample, both type granularities, the guard’s accounting (entries consulted, re-executions, whether the bucket was hit and whether its counterexample fired), token counts, and wall-clock split between model, oracle sandbox and guard sandbox. Episode summaries come from one function shared by every consumer, so “oracle calls to repair” cannot mean two things in two tables. Every model call is cached under a key containing the full prompt, model, temperature, output budget and the draw nonce (*task, seed, round*), which omits the arm and so implements common random numbers (Section 6); a cell therefore replays byte-for-byte and a stale entry cannot answer a different prompt. Episode identity is a deterministic hash of the experiment cell, so an interrupted run rewrites its own rounds instead of appending a truncated duplicate.

6 STUDY DESIGN

We ask five questions: whether memory reduces the cost of repair and how that varies with difficulty (RQ1); whether *typing* the memory beats a flat counterexample log, and whether any advantage is attributable to the types being right rather than to partitioning at all (RQ2); which operation does the work, guarding or steering (RQ3); whether an accepted patch is a repaired program and how that depends on oracle depth (RQ4); and whether the theory’s assumptions hold on real faults, including what typed steering costs when it misfires (RQ5).

6.1 Corpus and protocol

We use ConDefects [23], whose faults are AtCoder contest submissions from 2021–2023, curated to limit the training-data leakage that afflicts older benchmarks. Corpus construction is frozen and takes 526 examined candidates to 99: −64 whose reference disagreed with the shipped output, −48 too slow, −7 unresolved, −112 because a coding task may contribute one fault, and −196 never screened at all (Table 4). That last figure describes a sequential procedure honestly: candidates were screened at 40 model calls each and admitted until each band’s quota filled, so this is a quota sample, not a probability sample of a characterized pool. The 99 are a strict subset of a previously frozen 106, so nothing entered on the basis of results. The *oracle gate* admits a fault only if the reference reproduces the shipped outputs, at least one shipped case exposes the fault, and its natural mutants are caught—which is what makes Assumption 1 hold by construction. The *slow-reference filter* then drops a fault whose reference needs over 10 s on one of its first 20 cases. Its unit is the wrong one—cost is per *call*—and 45 of the 99 faults it kept still hold a call over that same threshold, the slowest 504 s. What it removed is *difficulty* (2042 against 940), a correlate of oracle cost rather than oracle cost; Section 9 measures which way that biases us.

The proposer is `qwen2.5-coder:7b (Q4_K_M`, served locally) at temperature 1.0, budget $B = 20$ proposals, oracle depth $k = 100$, 30 s sandbox timeout; Table 5 fixes the rest. Temperature is deliberately not lowered for determinism: the loop rests on redraw being meaningful, and at low temperature the same prompt returns approximately the same patch, so no-memory would loop forever on one wrong patch and redraw-after-block would be vacuous—every arm would degenerate rather than the comparison becoming cleaner. The cost is that the steering result is scoped to $T = 1.0$ (Section 9).

Tasks are banded by the proposer’s own per-round success rate $\hat{\pi}$, estimated from the no-memory arm run past its first accept so early stopping does not bias it: *dead* ($\hat{\pi} < 0.02$; 23 tasks), *hard* (17), *medium* (13), *easy* (25), *too-easy* (21), with pooled $\hat{\pi} = 0.207$ over 9,896 scored rounds. This is a *post hoc* covariate and not the design: the 40-call selection screen quantizes $\hat{\pi}$ to 0.025, so the *hard* band admits three attainable values and one success in forty separates it from *dead*. 42 of the 99 tasks report in a different band than they were selected into—*hard* 28 → 17, draining into *dead* 16 → 23—so the quota did not survive measurement, and per-band comparisons against no memory are inflated by regression to the mean. We report those columns as descriptive only; the typed-versus-untyped contrast within a band is unaffected, since under common random numbers the band comes from a draw the two arms share symmetrically.

6.2 Arms and pairing

Five arms switch the two memory operations independently: NO-MEM (both off, run past first accept for an unbiased $\hat{\pi}$), UNTYPED (unindexed guard, no prompt blocks), TYPED (indexed guard + steering), GUARD-ONLY and STEER-ONLY (one operation each). Four sweeps follow: oracle depth $k \in \{100, 20, 8, 3\}$; typing coherence $c \in$

Table 4: Corpus attrition, recomputed from the frozen screening record. Every filter is arm-independent and ran before any treatment. Two caveats travel with it. Screening was *sequential*—candidates were measured at 40 model calls each and admitted until a band’s quota filled—so the final row is a quota sample and the 196 were never screened, which is why no design weights exist for this corpus. And the slow-reference filter thresholds the *reference*’s per-case latency over the first 20 cases, while the cost that matters is per *call*: cases run times the slower of candidate and reference. That is why it left 45 of the 99 kept faults holding a call slower than its own 10 s threshold.

Stage	n	Criterion
Candidates examined	526	ConDefects Python faults
ineligible	−112	reference disagrees with the shipped output or raises (64); reference needs >10 s on one of its first 20 cases (48)
unresolved	−7	recorded as neither usable nor ineligible in the frozen counter
Usable	407	
not admitted to the pool	−112	at most one fault per coding task; pool size capped at 295
Screened pool	295	mutant gate: catches ≥ 0.67 of scoreable natural mutants; 295/295 passed
never screened	−196	sequential screening stopped once the band quotas filled: 115 targeted, 99 filled, the rest unmeasured
Frozen corpus	99	a strict subset of a previously frozen 106, so nothing entered on the basis of results

The 48 the slow-reference filter removed, against the 99 kept: reference worst case 10.02 s vs. 1.16 s, AtCoder rating 2042 vs. 940, shipped cases 44.2 vs. 38.9. It removes the faults whose oracle is most expensive—the regime the mechanism should help most—so our cost reductions understate what those faults would show, while our repair rates overstate it.

$\{1.0, 0.9, 0.75, 0.5, 0.25, 0.0\}$ plus a random-partition control; a redundancy audit that pays the oracle on blocked rounds to recover their classes; and a free-guarded-rounds condition (Corollary 3).

The draw nonce is $(task, seed, round)$ and *omits the arm*. Two arms whose prompt is byte-identical at a given round therefore receive the same completion, which pairs them on common random numbers [9] rather than buying the same draw repeatedly, and makes round 1 identical in every arm since memory is empty there. When prompts diverge, the prompt separates the cache keys, so nothing is shared that should not be. No sampling seed reaches the backend—the seed enters only the cache key—so a task’s five seeds are five *independent* draws at $T = 1.0$: round 1 shares their prompt yet returns 4.43 distinct patches of five on average, never one. Pairing is memoization, not a shared RNG, and it has a consequence we state plainly: the draw table is fixed once drawn, so all inference is *conditional* on it and the replication axis is tasks $\times$ seeds, not re-sampling of the model.

6.3 Metrics and statistics

Two outcomes on the three main arms are **confirmatory**: repair rate at a matched oracle-call budget, and oracle calls per

Table 5: Protocol. Everything above the rule is frozen across every arm and sweep; the response cache is keyed on the full prompt, so a change to any of it would invalidate the cache rather than silently mix two protocols.

Proposer	qwen2.5-coder:7b, Q4_K_M, served locally; temperature 1.0; context 32,768
Oracle	differential test over the shipped pool, depth $k = 100$ cases, 30 s per case
Type map	fine granularity: (edit line-range) $\times$ (violated property)
Budget	$B = 20$ proposals per episode
Corpus	99 ConDefects faults, frozen (\$6)
Seeds	1–5 main grid (495 cells/arm); 1–3 ablations and sweeps (297 and 90)
Scale	1,485 pre-specified main-grid cells, 0 missing; 3,654 episodes; 39,147 rounds
Pairing	common random numbers: draw nonce omits the arm

episode; everything else is exploratory. The primary metric is the fraction of cells repaired within b oracle calls, $b = 1 \dots 20$, pre-specified rather than total calls because an arm with no guard loses a total-call comparison *by construction*. On the budget curve nothing is guaranteed: spending fewer calls helps only if the calls spent still find repairs.

Secondary metrics are oracle calls; *total program executions and the seconds they take*—every run of a candidate against a case, whether the oracle asked for it or the guard did; *paid* and *caught* redundant verifications; entries consulted; tokens; and wall-clock. Redundancy is counted the same way in every arm: a blocked round carries no class of its own, so we recover it through the pairing from the byte-identical no-memory round the oracle already paid for. Where that recovery is unavailable—the steered arm, whose draws diverge—we report the censored fraction (52%).

Cells are paired on (*task, seed*); the default test is the exact two-sided sign test with $\hat{A}_{12}$ [19] for effect size, and because seeds of one task share a fault, every headline comparison is also reported per task with a task-clustered bootstrap interval over 10,000 resamples (Table 6). Sweep p -values are Benjamini–Hochberg adjusted [1]; audits observing zero failures get an exact one-sided Clopper–Pearson upper bound rather than a zero rate. Three falsifiers were pre-specified: a guarded arm must neither block a candidate the oracle accepts (Theorem 2) nor differ in repair rate from its unguarded twin (Corollary 3); redundancy *present* must be invariant across unconditioned arms, since they draw identically; and the random-partition control must *not* reproduce typing’s advantage.

7 RESULTS

All numbers come from the two frozen logs via the extractors in the artifact (Section 10); p -values are exact two-sided sign tests. Five seeds of one task share a fault, a model and a cache, so the cell is not an independent replication: Table 6 carries every headline comparison at both units with a task-clustered bootstrap interval, and the task-level result carries the claim.

The confirmatory family is repair at a matched oracle budget and oracle calls, on the three main arms, plus the two pre-specified criteria of the second-proposer study (Section 7.7); everything else is exploratory. Sections 7.1 to 7.6 are the qwen2.5-coder:7b grid; Section 7.7 repeats the main arms under gpt-4o-mini.

7.1 RQ1: what memory buys

Over all 495 paired cells of the 7B block of Table 6, memory cuts oracle calls per episode from 9.47 to 1.91 (untyped) and 1.83 (typed)— $\times 4.95$ and $\times 5.17$ —and *paid* redundant verifications, where a proposal whose class was already refuted still reaches the oracle, from 4.48 to 0.101 and 0.079. Repair rate is unchanged (0.703, 0.699, 0.697; flips 36/37, $p = 1.0$), so this is not accuracy traded for cost.

The guard executes, and the accounting says so. A guard that declines a validation still runs a stored counterexample, so a comparison denominated only in oracle calls would hide its cost. Ours does not: the execution counter bills every guard replay in every arm. Counting that way, total executions fall from 95.68 to 43.01, $\times 2.22$, on 251 of 364 discordant cells and 63 of 93 discordant tasks ($p = 0.0008$), with the guard’s share at 22%; in seconds the same comparison is $72.9 \rightarrow 38.2$, so the reduction is not an artifact of the unit. The split is conservative against the memory arms: a consultation is billed as an execution even when deduplication or memoization means no process started. Per round the trade is legible, and it is an ordering trade (Section 2.2): in TYPED a round the guard resolves costs 1.13 executions against 18.9 for one that reaches the oracle. Those surviving calls are dearer than no memory’s (12.5) because the cheap refutations are what the guard reaches first—a selection effect that flatters neither arm, since both totals are reported. Typing does *not* reduce total executions against a flat log (44.45 vs. 43.01; 170/146, $p = 0.20$); it reduces the time those consultations take. Table 8 gives both units per arm.

The saving is not spread evenly. One oracle call costs between 0.06 s and 110 s depending on the fault, and the reduction tracks that. In terciles of oracle expense, sandbox seconds fall $\times 1.00$, $\times 1.15$ and $\times 2.07$: where validation is cheap the guard’s replays cost about what they displace, so no time is saved at all. Executions fall in every tercile ($\times 1.68$ to $\times 2.66$) because that unit does not price a case (Table 7). The mechanism buys *time* only where validation is expensive, which is also what makes the slow-reference filter’s bias measurable (Section 9).

The primary metric. Figure 1 gives repair rate against a matched oracle-call budget. The arms coincide at $b = 1$ up to the timeout-edge cell. At $b = 2$ they separate—31.9% (no memory) against 51.3% (untyped) and 55.8% (typed)—and the memory arms are within 0.3 pp of their asymptote by $b = 4$, where no memory has reached 43.8%. No memory needs the full $b = 20$ to reach 70.3%; GUARD-ONLY reaches 70.7% at $b = 4$. The gain survives the conservative unit: the

Table 6: Main comparison under two proposers: 495 paired cells per proposer (99 faults $\times$ 5 seeds), means per episode, guard replays billed as program executions. The two blocks are separate studies with the same corpus, seeds, nonces and oracle; they are not pooled, because the proposer is a fixed factor and the contrast that differs between them (steering) is a finding. Because seeds of one task share a fault, a model and a cache, the per-cell sign test is anti-conservative: the test columns give the per-task test for the contrast named in the row, with a task-clustered bootstrap interval on the paired difference (10,000 resamples of the 99 tasks) where it is the claim; per-cell tests are in the supplement. Rows where TYPED is *worse* are marked, because two of them are findings.

	qwen2.5-coder:7b			gpt-4o-mini			per task	
	NO-MEM	UNTYPED	TYPED	NO-MEM	UNTYPED	TYPED	7b	4o-mini
Repair @ $B=20$	0.703	0.699	0.697	0.681	0.679	0.743	ty vs. un: parity	ty vs. un: 16/8, $p=0.15$
Repair @ $b=2$ calls	0.319	0.513	0.558	0.503	0.638	0.673	ty – no-mem [+18.4, +29.5] pp	[+11.5, +22.8] pp
Oracle calls / ep.	9.47	1.91	1.83	8.19	1.42	1.49	ty vs. un: 36/19, $p=0.030$	ty vs. un: 15/25, $p=0.15$
vs. NO-MEM		$\times 4.95$	$\times 5.17$		$\times 5.76$	$\times 5.48$	88/0, $p < 10^{-26}$; [−9.07, −6.27]	61/0, $p < 10^{-17}$; [−8.33, −5.14]
Program exec. / ep.	95.68	44.45	43.01	83.08	39.21	40.37	ty vs. un: 52/35, $p=0.086$	ty vs. un: 23/29, $p=0.49$
vs. NO-MEM		$\times 2.22$			$\times 2.06$		63/30, $p=0.0008$; [−84.1, −28.3]	40/20, $p=0.013$; [−70.1, −20.7]
Redundant <i>paid</i> / ep.	4.481	0.101	0.079	5.806	0.127	0.063	ty vs. un: 14/9, $p=0.40$	ty vs. un: 13/3, $p=0.021$
vs. NO-MEM		$\times 44.4$	$\times 56.9$		$\times 45.6$	$\times 92.7$	80/0, $p < 10^{-23}$; [−5.37, −3.52]	57/0, $p < 10^{-16}$; [−7.22, −4.35]
Entries consulted / ep.	0.00	10.20	9.38	0.00	8.20	7.16	ty vs. un: 48/39, $p=0.39$	ty vs. un: 32/21, $p=0.17$
Prompt tokens / ep.	4546	4592	6727	3649	3665	4374	ty vs. un: + 46% ; 22/75, $p < 10^{-7}$	ty vs. un: + 19% ; 21/43, $p=0.008$
Wall-clock s / ep.	108.8	77.6	140.5	85.1	54.3	49.5	ty vs. un: + 81% ; 41/58, $p=0.11$	ty fastest: −9%; 54/45, $p=0.42$
STEER-ONLY – NO-MEM, repair +2.0 pp, [−2.4, +6.4] on the 297 ablation cells (3 seeds) of each proposer, repair @ $B=20$ (Section 7.7).							11/8, $p=0.65$	21/3, $p=0.0003$

Table 7: Where the saving actually sits. The 99 kept faults, in terciles of how expensive one oracle call is on them—a property of the fault, measured on NO-MEM. Cost is counted to the first accepting round, as everywhere else. The corpus-level saving is concentrated in the expensive tercile, which is what makes the slow-reference filter’s bias measurable rather than assumed (Section 9). Per *fault* there is no rank association between oracle expense and reduction ($p = 0.00$): an expensive oracle enlarges the bill, it does not make an individual episode a better bet.

s / oracle call	n	sandbox s / ep.		executions / ep.	
		NO-MEM	→ TYPED	NO-MEM	→ TYPED
0.46 (cheapest third)	33	4.7	$\times 1.00$	54.8	$\times 1.68$
2.27	33	17.3	$\times 1.15$	99.0	$\times 2.14$
19.48 (dearest third)	33	196.7	$\times 2.07$	133.3	$\times 2.66$
all 99	99	72.9	$\times 1.91$	95.7	$\times 2.22$

task-clustered interval on the paired difference at $b = 2$ is [18.4, 29.5] pp.

Counted in *proposals* rather than oracle calls the arms nearly coincide (0.564 / 0.560 / 0.578 at eight charged proposals), exactly as Corollary 3 predicts when blocked rounds are charged: memory converts the expensive resource into repairs, it does not make the model smarter.

RQ1. Memory converts verification budget into repairs: $\times 5.2$ fewer oracle calls, $\times 2.2$ fewer program executions count-

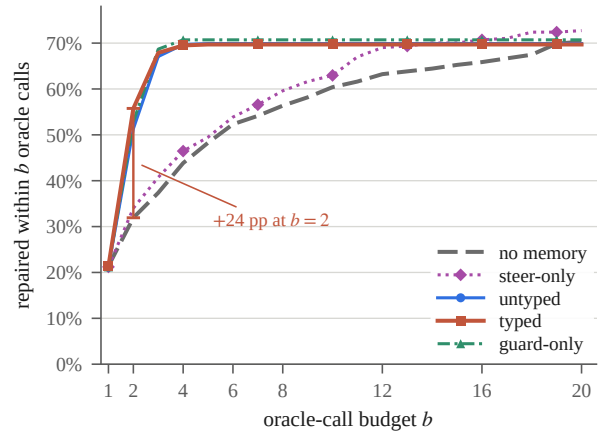


Figure 1: The pre-specified primary metric: repair rate at a matched *oracle-call* budget b , five arms, 495 main and 297 ablation cells. Memory saturates by $b = 4$; no memory needs $b = 20$ for the same rate. STEER-ONLY tracks no memory, which is Section 7.3’s finding in one line.

ing guard replays, and +23.8 pp repair at a two-call budget, at unchanged repair rate.

7.2 RQ2: does typing beat a flat log?

Typed uses fewer oracle calls than untyped on 93 of 152 discordant cells ($p=0.007$, $\hat{A}_{12} = 0.524$) and on 36 of 55 discordant tasks ($p=0.030$; signed-rank $p=0.023$), with a clustered in-

Table 8: All sandboxed work per episode—first as program executions, then as the seconds those executions took—split into the guard’s replays and the oracle’s checks. Both are counted to the first accepting round, the convention every cost number in the paper uses. The guard *does* execute, and this is the accounting in which it has to pay for itself. The split is conservative against the memory arms: a guard consultation is billed as an execution even when per-input deduplication or within-round memoization means no process was started, so their guard-side column is an upper bound.

Arm	cells	total	guard	oracle	guard %
NO-MEM	495	95.68	0.00	95.68	0
STEER-ONLY	297	92.41	0.00	92.41	0
UNTYPED	495	44.45	10.20	34.25	22.9
GUARD-ONLY	297	43.57	9.74	33.83	22.4
TYPED	495	43.01	9.38	33.64	21.8
<i>the same work in sandbox seconds</i>					
NO-MEM	495	72.91	0.00	72.91	0
STEER-ONLY	297	72.21	0.00	72.21	0
UNTYPED	495	41.49	7.11	34.38	17.1
GUARD-ONLY	297	32.36	4.04	28.31	12.5
TYPED	495	38.21	4.20	34.01	11.0

paired reductions in total executions

NO-MEM $\rightarrow$ TYPED: $\times 2.22$; 251/113 cells ($p < 10^{-12}$), 63/30 tasks ($p = 0.0008$); task-clustered CI on the paired difference $[-84.1, -28.3]$ executions.

UNTYPED $\rightarrow$ TYPED: $\times 1.03$; 170/146 ($p = 0.20$)—typing does *not* reduce total executions, only the time spent consulting memory.

terval on the paired difference of $[-0.141, -0.018]$ calls. On the 55 tasks of the three primary bands—the pre-specified universe of Section 7.7—the margin is 71/39 cells ($p = 0.0029$) and 28/12 tasks ($p = 0.017$), interval $[-0.24, -0.04]$ calls, and total program executions follow ($[-6.9, -0.9]$ per episode; Table 13). The task-level version is the claim: real, signed, and small—1.83 against 1.91 calls.

One caveat: TYPED differs from UNTYPED in two ways at once—indexing the guard by λ and putting exclusion blocks in the prompt—so the difference above is the package’s, and Section 7.3 separates them.

Is it the types, or just partitioning? Any typed index invites the objection that *any* partition would do, so the pre-specified control files each refutation under a location drawn uniformly from those seen so far—memory still partitions, but the partition carries no information about failure class. On the same 90 paired sweep cells exactly one quantity responds: the redundant proposals the guard *recognizes*, 4.80 at $c = 1.0$ against 4.47 for the control and 4.04 at $c = 0$, correct typing winning 9/9 ($p = 0.004$). Nothing else moves—paid redundancy ties on all 90; guard seconds, entries consulted, oracle calls and repair rate all give $p > 0.6$; and the bucket-only resolution rate is flat in c (33.1–35.3%) with the control inside it, because an uninformative partition still *fills* buckets.

RQ2. The licensed claim is narrower than “typing helps”: typing determines *which* redundant proposals the guard recognizes, does not affect blocking correctness (guaranteed by the fallback scan, Theorem 2), and on this sweep does not

Table 9: Headline comparisons at both units of analysis, with a task-clustered bootstrap interval on the paired difference (10,000 resamples of the 99 tasks). Seeds of one task share a fault, a model and a cache, so the cell-level test is anti-conservative and the task-level result carries each claim; every comparison keeps its sign and significance there. The supplement adds the remaining contrasts and budget points.

reference $\rightarrow$ treatment	per cell	per task	clustered CI
Oracle calls, NO-MEM $\rightarrow$ TYPED	325/10 $p < 10^{-80}$	88/0 $p < 10^{-26}$	$[-9.07, -6.27]$
Oracle calls, UNTYPED $\rightarrow$ TYPED	93/59 $p = 0.007$	36/19 $p = 0.030$	$[-0.141, -0.018]$
Program exec., NO-MEM $\rightarrow$ TYPED	251/113 $p < 10^{-12}$	63/30 $p = 0.0008$	$[-84.1, -28.3]$
Guard sec., UNTYPED $\rightarrow$ GUARD-ONLY	191/40 $p < 10^{-24}$	77/20 $p < 10^{-8}$	$[-6.47, -0.47]$
Prompt tokens, NO-MEM $\rightarrow$ STEER-ONLY	64/170 $p < 10^{-11}$	22/75 $p < 10^{-7}$	$[+4113, +7642]$
Repair @ $b=2$, gain over NO-MEM	0.319 $\rightarrow$ 0.558		$[+0.184, +0.295]$

measurably affect guard time.

7.3 RQ3: which operation does the work?

The guard: identical outcomes, cheaper answers. GUARD-ONLY and UNTYPED share their draws, so they should produce identical episodes and differ only in bookkeeping—and they do, on 295 of 297 cells, the two exceptions being the timeout-edge task (Section 7.8). The indexed arm repairs 0.707 against 0.704 and consults 9.74 against 10.02 (19/13, $p = 0.38$), while guard seconds fall $6.66 \rightarrow 4.04$ on 191 of 231 episodes ($p < 10^{-24}$). The *sign* is solid, the *magnitude* is not: the task-clustered interval is $[-6.5, -0.5]$ s and one fault supplies 1.8 s of the baseline (Appendix D). Holding the episode fixed isolates Proposition 7—the same work, answered faster—but the 35% of decisions the λ bucket settles is a rate the random-partition control also reaches, so the clock records deduplication, not the index.

Steering: null on every outcome, and expensive. STEER-ONLY keeps the prompt blocks and drops the guard. Round 1 matches no memory on all 297 cells and trajectories diverge where the first evidence block enters; after that nothing moves: repair $0.707 \rightarrow 0.727$ (23/17, $p = 0.43$), oracle calls $9.45 \rightarrow 8.81$ (80/71, $p = 0.52$), paid redundancy $4.43 \rightarrow 4.16$. What moves is the bill: prompt tokens $4519 \rightarrow 10,296$, +128% (64/170, $p < 10^{-11}$), growing +5.9 per round against untyped’s -1.2 .

The mechanism-level null is sharper than the outcome-level one. Length-matched by failed-round position, the rate at which the proposer re-draws an *already-eliminated* class is 50.2% without the exclusion block and 50.7% with it. The model conditions on the block—trajectories shift immediately—but does not avoid the classes it names. Equation (4)’s renormalization is requested and not granted.

The consequence is an ordering on the clock. Over the

Table 10: The two ablations, each paired against the arm it belongs against, on the same 297 cells. (a) isolates the *index*: the arms share their draws, so episodes should be identical and only bookkeeping may differ—and that is what happens. (b) isolates the *prompt blocks*, where nothing moves except the bill.

(a) The index: GUARD-ONLY vs. UNTYPED			
	UNTYPED	GUARD-ONLY	paired
Episodes	295/297		2 exceptions: the
outcome-identical			timeout-edge fault
Repair rate	0.704	0.707	the 2 timeout cells only;
			Corollary 3
Entries consulted /	10.02	9.74	19/13, $p=0.38$: the
ep.			same work
Guard seconds / ep.	6.66	4.04	−39%; 191/40; task CI
			[−6.5, −0.5] s
Wall-clock s / ep.	80.4	68.9	−14%; 238/57,
			$p < 10^{-26}$
(b) The prompt blocks: STEER-ONLY vs. NO-MEM			
	NO-MEM	STEER-ONLY	paired
Repair @ $B=20$	0.707	0.727	23/17, $p=0.43$: null
Oracle calls / ep.	9.45	8.81	80/71, $p=0.52$: null
Re-draw rate,	0.502	0.507	length-matched: <i>not</i>
eliminated class			<i>obeyed</i>
Prompt tokens / ep.	4519	10296	+128%; 64/170,
			$p < 10^{-11}$
Round-1 draw =	297/297		the pairing, verified
NO-MEM			

297 shared cells: GUARD-ONLY 68.9 s per episode, UNTYPED 80.4, STEER-ONLY 98.2, NO-MEM 110.8, TYPED 155.0. On a local 7B prompt length is latency, so typed’s prompt tokens become +93% wall-clock over untyped on these cells, and +81% on the main grid, where the paired test is 224 wins to 270 ($p=0.043$). The arm that dominates every other on wall-clock, oracle calls and tokens at once is GUARD-ONLY—the index without the prompt blocks.

Difficulty, and free guarded rounds. Two exploratory readings do not survive their tests and we report neither as a finding. Band-wise, the *dead* band suggested a steering bonus; a task-level interaction test (20,000 permutations of band labels) finds a difficulty dependence of the *whole* agent ($p=0.023$; *dead* +9.6 pp, *hard* -11.8 pp) and none for STEER-ONLY alone ($p=0.87$), so the gain is not the prompt’s. And charging a blocked round no budget unit lifts both memory arms’ repair (untyped 0.772 $\rightarrow$ 0.860, typed 0.815 $\rightarrow$ 0.889) while the guardless control is inert, but the flips come from 19 tasks and a task-level permutation gives $p=0.061$ (Table 11; Table 12).

RQ3. On this proposer the guard carries the value—identical episodes, cheaper guard time, 14% less wall-clock than untyped, no token overhead—while prompt-side steering moves no outcome metric at +128% prompt tokens. GUARD-ONLY is the configuration to deploy *at this scale*; Section 7.7 tests the qualifier.

7.4 RQ4: is an accepted patch a repair?

Every accepted patch is re-scored against the whole shipped pool. Because X_k exhausts that pool for 97 of the 99 faults at $k=100$ (Section 3), acceptance there usually already is

pool adequacy: all 1,256 audited acceptances pass with 0 overfitting, and 0 of the 939 carrying a regression verdict break a case the buggy version passed—the denominators differ because a regression verdict needs at least one such case and 317 acceptances belong to faults with none. A planted-mutant study finds 249/249 non-equivalent mutants caught (48 of 297 plants were equivalent). Three zero counts are not three zero rates: the one-sided 95% upper limits are 0.24%, 0.32% and 1.2%. What this establishes is membership in G_{pool} , not G^* .

Sweeping the depth shows why that matters. Raw acceptance *rises* as the oracle gets shallower, 0.678 at $k=100$ to 0.867 at $k=3$, and the audit says why: overfitting climbs 0%, 7.6%, 20.8%, 39.7% as the depth falls 100, 20, 8, 3. Full-pool repair is monotone, with *no difference* between the two deepest (0.678 at both 20 and 100, at 55% of the executions) while $k \leq 8$ is unsafe (0.633, then 0.522). Below $k=100$, X_k is a proper subset of most pools, and that is also where Theorem 2’s second clause is unaudited.

RQ4. At the reported depth acceptance means pool adequacy; a five-fold shallower oracle preserves it, and at $k=3$ two of five acceptances are false.

7.5 RQ5: assumptions, and what steering costs when it misfires

Informativeness measured, coherence not. An offline study re-executes stored counterexamples against other candidates of the same label: the cross-refutation rate is 0.908 (95% CI [0.884, 0.929]) at fine granularity and 0.822 [0.790, 0.852] at coarse, over 26,048 executions. This estimates ρ —how far a counterexample generalizes—which is what the guard’s usefulness depends on. It is *not* an estimate of c , the rate at which memory attributes a candidate to its true class; that needs a ground-truth labelling the corpus does not supply, so we perturb c directly instead (Section 7.2). Earlier drafts called this a coherence check; it is not, and only ρ is validated here.

The theory is directional, not calibrated. Fitting Theorem 6’s no-memory form across 79 tasks gives $r=0.657$ with relative MAE 1.33, an order-of-magnitude prediction the data respects in shape; the memory-arm form could not be fit at all, its predicted spread being smaller than the sampling noise. Their i.i.d. assumption is what a real proposer violates, and the violation is the redundancy a guard exists to intercept (Section 2.3).

Anchoring: the cost of excluding a class. Typed steering can exclude the class the *correct* fix belongs to. Over 1,503 steered episodes at $c=1.0$ the correct patch’s edit location had entered the exclusion block in 5.7% of episodes that then failed, and the rate is highest in exactly the band where steering helps most (19.7% in *dead*, 0 in the two easiest). Steering is a high-variance intervention worth making only where the baseline is near zero.

7.6 Against the mechanism the guard resembles

Section 2.2 puts the guard in the same row as fault-recorded prioritization [16] and its modification-point-aware refine-

Table 11: Cost by difficulty band, banded on the no-memory arm’s own $\hat{\pi}$. The NO-MEM column is descriptive only (Section 6); inference is confined to the last column, where the band is symmetric between the two memory arms.

Band	tasks	oracle calls / ep.				TYPED vs.	
		NO-MEM	UNTYPED	TYPED	×	UNTYPED	
dead	23	19.83	1.62	1.64	12.1	18/19, $p=1.0$	
hard	17	13.81	2.32	2.14	6.5	32/17, $p=0.044$	
medium	13	7.66	2.28	2.19	3.5	15/10, $p=0.42$	
easy	25	4.47	2.11	1.98	2.3	24/12, $p=0.065$	
too-easy	21	1.71	1.45	1.42	1.2	4/1, $p=0.38$	
<i>pooled over the three primary bands (275 cells): 71/39, $p=0.0029$</i>							
<i>repair rate in the dead band, where steering acts (Section 7.3)</i>							
NO-MEM / UNTYPED		0.026 / 0.026	(identical: the guard cannot change an outcome)				
TYPED		0.122	(per-task $p=0.047$ vs. UNTYPED)				

Table 12: Corollary 3, tested. A blocked round costs a model call but no unit of the proposal budget, so the attempts the guard saves are handed back to the search. Both memory arms gain repair at unchanged oracle cost; the no-memory control, having no guard, is inert—which is what makes the gain attributable to the guard rather than to the extra proposals.

Arm	cells	repair	oracle	props.	flips
NO-MEM chg.	60	0.800	8.62	8.62	0/1
NO-MEM free	60	0.783	8.60	8.60	
UNTYPED chg.	57	0.772	2.12	9.30	5/0, $p=0.06$
UNTYPED free	57	0.860	2.09	14.56	
TYPED chg.	54	0.815	2.02	7.65	4/0, $p=0.13$
TYPED free	54	0.889	2.09	13.44	
<i>pooled over the two memory arms:</i>					9/0, $p=0.004$ (suggestive: shared tasks)

ment [20]. We measure the distance rather than argue it, and exactly: in NO-MEM the candidate stream depends on task, seed and round alone, so all five policies replay over byte-identical candidates against a complete verdict matrix—495 cells, 99 tasks, no model call (Appendix B).

The pre-specified criterion fails, on an axis with little to win. We fixed beforehand that CEGMEM-GUARD would reach a refuting case in no more executions than the better classical policy on at least 60 of the 99 tasks. It does so on 1, ties on 18, loses on 80 ($p=1.0$), and loses on total executions too (2/6/91 tasks, 179.7 against MP-AWARE’s 173.7 per episode). The margin is a separate question: the median task pays +4.6% more, a gap worth 1.1% of the headroom this corpus affords at all (Section 9). The sign is not in doubt; the size is small.

What no prioritizer does. Both classical policies leave oracle invocations at 19.99 per episode: they change the order in which a validation seeks evidence, not whether it happens. The guard resolves 70.8% of rounds before the oracle is reached, taking invocations to 5.85—99 of 99 tasks, $p < 10^{-17}$ —with accepted rounds identical across all five policies, so none of it is bought with repair. Nor does the index earn anything here: CEGMEM-GUARD and DEDUP-GUARD block the identi-

cal rounds on every one of the 495 cells.

7.7 RQ6: does the answer survive a second proposer?

Every number so far is the left block of Table 6. Section 9 predicted that the guard result transfers, being a property of the runtime, and that the steering null might not, being a property of what a model does with an instruction. Both predictions were fixed as a falsifier before the run (Section 9): on a second proposer, over the 55 tasks the frozen banding places in the primary bands, (a) the guard’s oracle-call reduction must be at least $\times 3.0$, and (b) the task-clustered interval on the STEER-ONLY repair difference must contain zero. We ran the three main arms and STEER-ONLY under gpt-4o-mini, same corpus, same seeds, same nonces, same oracle, and the same pairing—memoization on prompt and nonce, so round 1 is one draw shared by every arm.

The guard transfers. Criterion (a) holds with room: $\times 4.93$ (untyped) and $\times 4.82$ (typed) on the pre-specified 55 tasks, $\times 5.76$ and $\times 5.48$ on all 99 (right block of Table 6), against $\times 5.17$ at 7B. Every task moves the same way (59/0 and 61/0, $p < 10^{-17}$). Program executions fall $\times 2.06$, paid redundant verifications $\times 92.7$ —the largest fold in either study—and repair is unchanged between NO-MEM and UNTYPED (0.681 vs. 0.679). What the 7B study attributed to the runtime, the second proposer confirms is the runtime’s.

The steering null does not. Criterion (b) fails. On the 55 pre-specified tasks STEER-ONLY repairs 0.794 against NO-MEM’s 0.727, a task-clustered interval of $[+0.6, +12.7]$ pp that excludes zero, 13 tasks up and 2 down ($p=0.007$); on all 99 it is +9.1 pp, $[+4.0, +14.5]$, 21/3 ($p=0.0003$). The same arm on the same 55 tasks under the 7B proposer gives +1.2 pp, $[-4.8, +7.3]$ —the null Section 7.3 reports. The prompt half is therefore not inert; it is inert *for a proposer that does not act on it*. The cost is unchanged in kind: +67% prompt tokens for STEER-ONLY, +19% for the full TYPED (+9% on the primary-band tasks, interval including zero), which now repairs 0.743 against UNTYPED’s 0.679 (46/14 cells, $p < 10^{-4}$; 16/8 tasks, $p=0.15$; CI $[+1.8, +11.7]$ pp). That gain is not a

Table 13: Table 6 restricted to the pre-specified primary-band universe: the 55 tasks the frozen banding places in *hard*, *medium* or *easy* (275 paired cells per arm per proposer; same accounting, same tests). This is the universe the second-proposer criteria were fixed on, and it is where a repair-rate difference *can* appear—the *dead* and *too-easy* bands are at floor and ceiling. Restricting to it sharpens the typed-versus-untyped oracle-call contrast at 7B and shrinks the guard’s fold in both blocks, because the *dead* band, where the guard saves most, is excluded; both movements are reported.

	gwen2.5-coder:7b			gpt-4o-mini			per task	
	NO-MEM	UNTYPED	TYPED	NO-MEM	UNTYPED	TYPED	7b	4o-mini
Repair @ $B=20$	0.873	0.865	0.822	0.738	0.735	0.785	ty – un: -4.4 pp, [-10.9, +2.2]; 9/15, $p=0.31$	ty – un: +5.1 pp, [+0.0, +10.9]; 9/4, $p=0.27$
Repair @ $b=2$ calls	0.262	0.560	0.589	0.524	0.684	0.695	ty – no-mem [+24.7, +40.7] pp	[+9.5, +25.1] pp
Oracle calls / ep.	8.11	2.22	2.08	7.34	1.49	1.52	ty vs. un: 28/12, $p=0.017$; [-0.24, -0.04]	ty vs. un: 9/12, $p=0.66$; [-0.04, +0.11]
vs. NO-MEM		×3.66	×3.91		×4.93	×4.82	54/0, $p<10^{-15}$; [-7.26, -4.85]	35/0, $p<10^{-10}$; [-7.84, -3.88]
Program exec. / ep.	68.34	49.68	45.98	56.34	39.78	40.32	ty vs. un: 34/21, $p=0.11$; [-6.9, -0.9]	ty vs. un: 13/16, $p=0.71$; [-0.8, +2.0]
vs. NO-MEM		×1.38	×1.49		×1.42	×1.40	38/15, $p=0.0022$; [-35.1, -11.9]	20/13, $p=0.30$; [-29.4, -5.3]
Redundant <i>paid</i> / ep.	3.415	0.113	0.102	4.916	0.149	0.080	ty vs. un: 7/8, $p=1.0$	ty vs. un: 8/2, $p=0.11$; [-0.14, -0.01]
vs. NO-MEM		×30.3	×33.5		×33.0	×61.5	49/0, $p<10^{-14}$; [-4.18, -2.49]	30/0, $p<10^{-8}$; [-6.58, -3.16]
Entries consulted / ep.	0.00	8.97	8.27	0.00	7.46	6.13	ty vs. un: 25/30, $p=0.59$	ty vs. un: 17/13, $p=0.59$; [-2.68, -0.13]
Prompt tokens / ep.	4427	4510	6421	3738	3763	4104	ty vs. un: +42%; 18/37, $p=0.015$	ty vs. un: +9%; 13/23, $p=0.13$; [-507, +1151]
Wall-clock s / ep.	85.3	91.6	142.4	59.2	56.9	55.2	ty vs. un: +55%; 25/30, $p=0.59$	ty vs. un: -3%; 29/26, $p=0.79$
STEER-ONLY – NO-MEM, repair +1.2 pp, [-4.8, +7.3] on the 165 ablation cells (3 seeds) of each proposer that fall in the primary bands, repair @ $B=20$ (criterion F7(b)).								
				+6.7 pp, [+0.6, +12.7]			8/6, $p=0.79$	13/2, $p=0.0074$

band artefact: on the frozen banding every band’s typed-minus-untyped repair difference is non-negative (*dead* +12.2 pp, *hard* +10.6 pp, *medium* 0.0, *easy* +4.0 pp, *too-easy* +3.8 pp), and the task-level interaction test that found a two-signed profile for this contrast at 7B ($p=0.03$) finds none here ($p=0.58$); on the 55 primary-band tasks the difference is +5.1 pp, [+0.0, +10.9] (Table 13).

What did not change. The index still buys no oracle calls (TYPED 1.49 vs. UNTYPED 1.42; 15/25 tasks, $p=0.15$). Typing does show on the one axis Section 7.2 licensed for it: TYPED pays for fewer redundant verifications than UNTYPED on 13/3 tasks ($p=0.021$, CI [-0.11, -0.02]), a contrast that was signed but not significant at 7B. And the wall-clock ordering inverts for the reason Section 7.3 gave: on a hosted API prompt length is not latency, so TYPED is the fastest arm (49.5 s) rather than the slowest.

RQ6. The guard is model-independent infrastructure: ×4.8–×5.8 fewer oracle calls at unchanged repair on both proposers. Steering is a model-capability question: null at 7B, +7 pp to +9 pp repair at +67% tokens on gpt-4o-mini. The pre-specified failure rule applies and the recommendation narrows to *the guard unconditionally; steering where the proposer follows it*.

7.8 Integrity of the comparison

Common random numbers hold exactly: round 1 is byte-identical to the no-memory arm on 495/495 main-grid and 297/297 ablation cells in all four treatment arms, the frozen grid has 0 missing cells of 1,485, and the three pre-specified falsifiers behave. A single fault whose candidates run at the 30 s sandbox edge produces every cross-arm inconsistency in the study, including the two non-identical guard-only episodes. We left the frozen timeout unchanged and repeat the headline numbers without it in Appendix D.

8 DISCUSSION

The two halves of typed memory depend on different things, and that is why one transfers across proposers and the other does not. Obeying Eq. (4) requires the proposer to simulate the oracle’s verdict on a class described to it only in the abstract, which is harder than writing a patch and is a capability the proposer either has or lacks; the guard needs no such simulation, since it re-executes a concrete counterexample and reads the answer, and so owes nothing to the proposer. Hence the 7B model *conditions* on the exclusion block—trajectories diverge the moment it appears—while re-drawing the excluded classes at an unchanged rate, as the concurrent text-to-SQL work [21]

Table 14: Five validation policies replayed over one candidate stream, against a complete candidate $\times$ case verdict matrix, with no model call. The replay walks all 20 rounds instead of stopping at the first acceptance—which is what keeps the stream policy-independent—so the levels are not Table 6’s and only within-table comparisons are meant. Executions and accepts are per-episode means; *to refut.* is the median cell’s executions spent reaching a refuting case. The last column is identical across all five by construction: a policy changes what finding out costs, not what the proposer writes. Task-level comparisons are in Section 7.6.

Policy	Exec/ep	To refut.	Oracle/ep	Accepts/ep
ORACLE- <i>k</i> (reported)	248.4	2.91	19.99	4.10
Q1’13	174.1	1.44	19.99	4.10
MP-AWARE	173.7	1.38	19.99	4.10
DEDUP-GUARD	180.1	1.54	5.85	4.10
CEGMEM-GUARD	179.7	1.50	5.85	4.10

also reports; and hence `gpt-4o-mini`, which can act on the block, converts it into repair (Section 7.7). A class label is worth more as an *index over evidence* than as an *instruction to a model* unless the model is one that follows instructions of that kind; the design consequence is to count oracle calls and executions rather than tokens, since the resource identified as squandered [10] is invisible in a token budget.

9 THREATS TO VALIDITY

Construct validity. Our primary metric counts *oracle calls*, the resource recent measurement identifies as squandered [5, 10]. It is not the only cost model—an agent billed per token, or one whose suite is cheap, would rank the arms differently, and our own wall-clock row shows the full typed agent losing under a latency-weighted one—so we report oracle calls, program executions, tokens and wall-clock separately and name the resource each claim is about. Guard replays are billed as executions and we claim no avoided execution: a full X_k validation is replaced by one that terminates on its first case, an ordering effect (Section 2.2) that survives the accounting. And “repair” is acceptance, a claim about G_k rather than a fact about G^* ; we audit it against G_{pool} with exact bounds, and no test-based procedure can certify G^* .

Internal validity. Common random numbers make the comparison tight but make all inference *conditional on a fixed draw table*: the replication axis is tasks $\times$ seeds, not re-sampling of the model. Difficulty bands come from the no-memory arm’s own $\hat{\pi}$, so per-band comparisons against no memory are inflated by regression to the mean and are descriptive only. The timeout-edge fault of Section 7.8 accounts for every cross-arm inconsistency; removing it leaves the cost and repair results unmoved but cuts the guard-second reduction from -39% to -18% (Appendix D), so that magnitude rests on one fault and we do not lean on it. Typed’s *caught*-redundancy count is 52%

Table 15: Integrity of the comparison. The first block is the pairing the study rests on; the second is the three falsifiers we pre-specified, each of which would have invalidated a claim had it failed.

Check	Result
Round-1 draw identical to NO-MEM	495/495 (UNTYPED, TYPED); 297/297 (GUARD-ONLY, STEER-ONLY)
Frozen grid complete Proposer constant	1,485 of 1,485 cells, 0 missing one model string in every row of every arm
Cache replay exact	same key $\Rightarrow$ same patch, no exception
<i>F1</i> : a guarded arm’s repair rate equals its unguarded twin’s (Corollary 3)	holds: 0.704 vs. 0.707; 295/297 episodes outcome-identical
<i>F2</i> : redundancy <i>present</i> invariant across unconditioned arms	holds: 4.48 / 4.50 (NO-MEM, UNTYPED)
<i>F3</i> : the random partition must <i>not</i> reproduce typing’s advantage	holds on the responding column (9/0, $p=0.004$); indistinguishable on the rest

censored, because a blocked round in the steered arm has no paired no-memory draw to recover its class from; the claim in Table 6 rests on *paid* redundancy, uncensored in every arm.

External validity. Scope is the strongest limitation. *Two proposers, one temperature:* the main grid is `gwen2.5-coder:7b` at $T=1.0$ and the main arms are repeated under `gpt-4o-mini` (Section 7.7). That repeat tested two predictions fixed before it ran: that the guard result transfers, being a property of the runtime, and that the steering null might not, being a property of what a proposer does with an instruction. Both held—the guard’s fold is $\times 4.8$ – $\times 5.8$ on both, and steering is null at 7B and +9 pp on `gpt-4o-mini`—so the recommendation is scoped by proposer rather than stated once. Two proposers are not a scaling law; we make no claim about where between them, or beyond them, steering starts to pay. The second run also shows why the corpus’s bands do not travel: under `gpt-4o-mini` 47 of 99 faults sit in the *too-easy* band and 29 in *dead*, leaving 23 in the primary bands where the frozen banding placed 55 (Section 6), so per-band comparisons on the second proposer are descriptive only. *One fault distribution, and a filtered one:* ConDefects faults are single-file competitive-programming submissions with a rich shipped pool, which is what makes a clean counterexample oracle possible and is unlike a repository-scale issue. The slow-reference filter thinned the harder faults; rather than assert which way that biases us we measure it inside the kept corpus, where oracle cost still spans a factor of 1,800: the saving in sandbox seconds is $\times 1.00$ in the cheapest third of faults and $\times 2.07$ in the dearest (Section 7.1), so the aggregate sits in exactly the regime the filter thinned, and the dropped faults would have deepened it rather than reversed it. Because the corpus is a quota sample (Section 6), none of this is a population estimate; the paired design is what carries the claims. *One*

scale of memory: episodes run at most 20 rounds and memory holds a median of one entry when the guard fires, which is why we claim nothing for the index.

Statistical validity, and what the baseline comparison can bear. A per-cell test overstates precision, so every headline comparison is reported at both units with a task-clustered interval (Table 6); all keep their sign there, with typed-versus-untyped weakening from $p=0.007$ to $p=0.030$ as it should. Only two outcomes on three arms, and the two second-proposer criteria of Section 7.7, are confirmatory; one of those criteria failed and we report the failure as the finding it is. The dead-band result now has the interaction test it lacked (Section 7.3), and the test does not support the reading we first gave it. What survives is a difficulty dependence of the *whole* agent, one band up and one down on 23 and 17 tasks, which we report as exploratory. Finally, Table 1 is now measured: both classical prioritizers run over byte-identical candidates (Section 7.6) and the pre-specified criterion failed. Two caveats bound that verdict. The replay sits in the NO-MEM universe at full horizon, so a prioritizer driving a *steered* proposer is a different experiment. And the ordering axis is nearly saturated here: a random order already refutes in ≈ 1.7 executions against an ideal order’s 1.0, and every informed policy we ran falls between 1.38 and 1.54 (Appendix B).

Reproducibility, and what “pre-specified” means. We say *pre-specified* rather than pre-registered: metrics, arms, falsifiers and the banding rule were fixed in a design document and in the frozen corpus and protocol artifacts before the reported run, timestamped in the archive, but there is no third-party registry entry. Two chronologies bear on that. The guard correction of Section 5 was made after inspecting an *earlier* run’s accounting and *before* the reported run. And the two second-proposer criteria were committed before that run started, but the proposer itself changed: the register named a 14B model of the same family, and availability substituted `gpt-4o-mini`, so the criteria were fixed but the model they were applied to was not.

10 CONCLUSION

LLM repair agents run tests indiscriminately and most of that execution does not pay [5, 10]. Classical repair cuts that work either by relating candidate patches to one another [12, 22], which an unenumerable candidate space rules out, or by ordering the evidence so a doomed candidate dies on the first case [16, 20]. This paper carries the second mechanism into an LLM loop and changes what it orders from: counterexamples retained across candidates, where an input already in memory *proves* that a candidate reproducing it will fail. Memory cuts oracle calls $\times 5.2$ at unchanged repair rate and, counting every guard replay as the execution it is, total program executions $\times 2.2$. That there is anything to intercept is itself a finding: one proposal in four is byte-identical to an earlier one, a rate that is zero by construction in the synthesis loop the design came from.

The two halves of the architecture depend on different things, and a second proposer shows which. The guard is a property of the runtime: under `gpt-4o-mini` it cuts oracle calls $\times 5.8$ and paid redundant verifications $\times 46$ on its own ($\times 93$ with steering), again at unchanged repair, and its λ *ordering* is again indistinguishable from a flat scan on memories this small. Steering is a property of the proposer: null at 7B for $+128\%$ prompt tokens, worth $+9$ pp repair for $+67\%$ tokens on `gpt-4o-mini`. What we recommend is therefore the guard unconditionally and steering where the proposer acts on it. The guard does not survive a per-modification-point prioritizer on executions—that prioritizer is cheaper on 80 of 99 tasks, by a median 4.6%—and it is the only policy of five that removes oracle invocations, on every task (Section 7.6). Where between two proposers steering begins to pay, and whether any of this holds on the expensive-oracle faults we filtered out, are the two questions we would want answered next.

DATA AVAILABILITY

One archive holds the source, the two frozen episode logs (39,147 rounds under `qwen2.5-coder:7b`, 19,768 under `gpt-4o-mini`), the corpus and banding artifacts, the prompt templates, a pinned model manifest for the local proposer and the API model identifier and call dates for the hosted one, the lockfile, the response caches, the pre-specification register with its commit timestamps, and the six extractors that regenerate every number here from them without a GPU, plus the verdict matrix behind Section 7.6 and the checker that re-derives that table from it. *Access*: [anonymized DOI].

REFERENCES

- [1] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995. doi: 10.1111/j.2517-6161.1995.tb02031.x.
- [2] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. RepairAgent: An autonomous, LLM-based agent for program repair. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2188–2200, 2025. doi: 10.1109/ICSE55347.2025.00157.
- [3] Lingchao Chen, Yicheng Ouyang, and Lingming Zhang. Fast and precise on-the-fly patch validation for all. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 1123–1134, 2021. doi: 10.1109/ICSE43902.2021.00104.
- [4] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [5] Zhi Chen, Zhensu Sun, Yuling Shi, Chao Peng, Xiaodong Gu, David Lo, and Lingxiao Jiang. Rethinking the value of agent-generated tests for LLM-based software engineering agents. *arXiv preprint arXiv:2602.07900*, 2026.

- [6] Ryan Ehrlich, Bradley Brown, Jordan Juravsky, Ronald Clark, Christopher Ré, and Azalia Mirhoseini. CodeMonkeys: Scaling test-time compute for software engineering. *arXiv preprint arXiv:2501.14723*, 2025.
- [7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [8] René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*, pages 437–440. ACM, 2014. doi: 10.1145/2610384.2628055.
- [9] Averill M. Law and W. David Kelton. *Simulation Modeling and Analysis*. McGraw-Hill, Boston, MA, 3rd edition, 2000. ISBN 0070592926. Common random numbers: Chapter 11, Section 11.2.
- [10] Zhihao Lin, Junhua Zhu, Mingyi Zhou, Xin Wang, Zhensu Sun, Renyu Yang, David Lo, and Li Li. To run or not to run: Analyzing the cost-effectiveness of code execution in LLM-based program repair. In *Proceedings of the 35th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2026. doi: 10.48550/arXiv.2606.26978. To appear.
- [11] Yiling Lou, Jun Yang, Samuel Benton, Dan Hao, Lin Tan, Zhenpeng Chen, Lu Zhang, and Lingming Zhang. When automated program repair meets regression testing—an extensive study on two million patches. *ACM Transactions on Software Engineering and Methodology*, 33(7): 1–23, 2024. doi: 10.1145/3672450.
- [12] Sergey Mechtaev, Xiang Gao, Shin Hwei Tan, and Abhik Roychoudhury. Test-equivalence analysis for automatic patch generation. *ACM Transactions on Software Engineering and Methodology*, 27(4):1–37, 2018. doi: 10.1145/3241980.
- [13] Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, 2026. To appear.
- [14] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [15] Pedro Orvalho, Mikoláš Janota, and Vasco M. Manquinho. Counterexample guided program repair using zero-shot learning and maxsat-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 649–657, 2025. doi: 10.1609/aaai.v39i1.32046.
- [16] Yuhua Qi, Xiaoguang Mao, and Yan Lei. Efficient automated program repair through fault-recorded testing prioritization. In *2013 IEEE International Conference on Software Maintenance (ICSM)*, pages 180–189, 2013. doi: 10.1109/ICSM.2013.29.
- [17] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, pages 8634–8652, 2023. Author list follows the NeurIPS camera-ready.
- [18] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit A. Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 404–415. ACM, 2006. doi: 10.1145/1168857.1168907. Introduces the counterexample-guided synthesis loop; the term “CEGIS” does not appear in the paper.
- [19] András Vargha and Harold D. Delaney. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics*, 25(2):101–132, 2000. doi: 10.3102/10769986025002101.
- [20] Yazhini Venugopal, Phung Quang-Ngoc, and Lee Eun-seok. Modification point aware test prioritization and sampling to improve patch validation in automatic program repair. *Applied Sciences*, 10(5):1593, 2020. doi: 10.3390/app10051593.
- [21] Khang Nhat Hoang Vo, Tam Minh Chu, Anh Trac Duc Dinh, Thuyen Vinh Ha Bui, and Tho Quan. Causal episodic memory for feedback-driven agent repair. *arXiv preprint arXiv:2608.05906*, 2026.
- [22] Westley Weimer, Zachary P. Fry, and Stephanie Forrest. Leveraging program equivalence for adaptive program repair: Models and first results. In *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 356–366, 2013. doi: 10.1109/ASE.2013.6693094.
- [23] Yonghao Wu, Zheng Li, Jie M. Zhang, and Yong Liu. ConDefects: A complementary dataset to address the data leakage concern for LLM-based fault localization and program repair. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion)*, pages 642–646. ACM, 2024. doi: 10.1145/3663529.3663815.
- [24] Chunqiu Steven Xia and Lingming Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. In *Proceedings of*

the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pages 819–831. ACM, 2024. doi: 10.1145/3650212.3680323.

- [25] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi: 10.1145/3715754. The arXiv version is titled “Agentless: Demystifying LLM-based Software Engineering Agents” (arXiv:2407.01489).
- [26] Yuan-An Xiao, Chenyang Yang, Bo Wang, and Yingfei Xiong. Accelerating patch validation for program repair with interception-based execution scheduling. *IEEE Transactions on Software Engineering*, 50(3):618–635, 2024. doi: 10.1109/TSE.2024.3359969.
- [27] Yingfei Xiong, Xinyuan Liu, Muhan Zeng, Lu Zhang, and Gang Huang. Identifying patch correctness in test-based program repair. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pages 789–799. ACM, 2018. doi: 10.1145/3180155.3180182.
- [28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, pages 50528–50652, 2024.
- [29] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1592–1604. ACM, 2024. doi: 10.1145/3650212.3680384.

A STATEMENTS AND PROOFS

Section 4 states Assumption 1, Theorem 2, and Corollary 3 in full and summarizes the three results below; this appendix states those three formally and proves all of them. Two further assumptions are used here and nowhere in the body.

Assumption 2 (Ideal typing). $\rho = c = 1$ (Definition 1) and refutation is *class-exclusive*: a counterexample of class τ refutes every candidate of class τ , and no candidate of any other class.

The exclusivity clause is what makes “a round calls the oracle iff its type is not yet eliminated” an equivalence rather than an implication, and both results below need it: without it a live-type proposal can be blocked by a foreign counterexample, eliminating nothing and consuming a round, which destroys the proposal bound while leaving the call bound intact. It is also the assumption our implementation most clearly violates— λ records where a candidate edits and refutation is decided by which input it gets wrong, and Section 5 reports the two

are close to independent—so Theorems 4 and 6 are directional predictions rather than calibrated ones, and Section 7.5 reports how far the fit holds.

Assumption 3 (Memoryless proposer). The type of each proposal is drawn i.i.d. with base mass $w_\top = \pi > 0$ on the correct class and $w_\tau = q_\tau$ on each failure type; steering (Eq. (4)) modifies the support but not the relative weights. $K = |\{\tau \in \mathcal{T} : q_\tau > 0\}|$ is the number of *reachable* types—the only ones a run can eliminate.

Assumption 3 is the conservative modeling choice: it endows the proposer with no intrinsic learning, so every improvement proved below is attributable to memory alone. It is also close to what a 7B proposer does—the measured rate at which it re-draws an already-eliminated class is statistically indistinguishable with and without an exclusion instruction (Section 7.3).

Theorem 4 (Non-repetition and round bound). *Under Assumptions 1, 2 and 3, on an unbounded run with typed memory: (i) the agent never proposes a candidate whose class is already eliminated, so no refuted patch is re-proposed; (ii) every refuting oracle call eliminates a previously live reachable class, so the run halts within $K+1$ oracle calls and $K+1$ proposals. Untyped memory attains the same call bound but no proposal bound; no memory attains neither, issuing $1/\pi$ calls in expectation and unbounded in the worst case.*

Three budget quantities stay distinct: oracle calls, generated proposals, and *charged* proposals (rounds billed against B , including blocked ones). Theorem 4 bounds the first two on an unbounded run; under a finite B it becomes the statement that the run repairs if $K+1 \leq B$. When K exceeds the budget, untyped memory can spend arbitrarily many charged blocked rounds and stop at B without reaching $K+1$ oracle calls.

Definition 5 (Observed-type history). $E_t^{\text{obs}} := \{\theta(p_u, x_u) : u < t, \mathcal{O}_k(p_u) = x_u\}$ is the set of classes already refuted this episode before round t , whether or not the arm records them. Under Assumption 2 typed memory has $E = E^{\text{obs}}$; untyped memory and no memory have $E = \emptyset$ while E^{obs} is unchanged. Proposal p_t is a *redundant attempt* if $\theta(p_t, \cdot) \in E_t^{\text{obs}}$; it is *caught* if the guard blocks it and *paid* if it reaches the oracle. Defining redundancy this way is what makes it measurable in an arm that stores nothing.

Theorem 6 (Redundancy reduction). *Under Assumptions 1, 2 and 3, on an unbounded run, with $D := \sum_\tau q_\tau / (q_\tau + \pi)$: (a) expected oracle calls are $1/\pi$ with no memory and $1 + D$ with either memory, and since $\frac{1}{\pi} - (1 + D) = \sum_\tau q_\tau (\frac{1}{\pi} - \frac{1}{q_\tau + \pi}) \geq 0$, memory strictly reduces them whenever some $q_\tau > 0$; (b) with E^{obs} as in Definition 5, expected redundant attempts generated are $1/\pi - 1 - D > 0$ with no memory and with untyped memory and 0 with typed memory, while those paid are that same quantity with no memory and 0 with either memory.*

Proposition 7 (Index position and cost). *Fix a round on which memory holds m entries at insertion positions $1, \dots, m$, and let R be the positions of the entries that still refute the candidate.*

A flat scan performs $\min R$ re-executions, or m if $R = \emptyset$. Write $\mathcal{B} = \mathcal{M}[\lambda(p)]$ for the bucket and $\text{pos}_{\mathcal{B}}(j)$ for a position's rank within it. The bucket-first guard performs $\min\{\text{pos}_{\mathcal{B}}(j) : j \in R \cap \mathcal{B}\}$ when $R \cap \mathcal{B} \neq \emptyset$; otherwise it performs $|\mathcal{B}| + \min R$, or m when $R = \emptyset$. Writing h for the probability that R meets $\mathcal{B}$, expected re-executions therefore fall by the saving h buys inside the bucket less $(1-h)|\mathcal{B}|$ paid on a miss; the index is never asymptotically cheaper. Deduplicating stored counterexamples by input and memoizing verdicts within a round reduce executions at unchanged consultations.

Lemma 8 (Competing-geometric race). *Under Assumption 3, the expected number of distinct failure types the proposer draws before its first correct proposal is $D = \sum_{\tau \in \mathcal{T}} q_{\tau} / (q_{\tau} + \pi)$.*

Proof. Fix τ . Ignore every draw whose type is neither τ nor $\top$; the remaining subsequence is i.i.d. with $\mathbb{P}[\tau] = q_{\tau} / (q_{\tau} + \pi)$ and $\mathbb{P}[\top] = \pi / (q_{\tau} + \pi)$. Type τ is drawn at least once before the first correct proposal iff the first element of that subsequence is τ , which has probability $q_{\tau} / (q_{\tau} + \pi)$. Summing the indicator over $\tau \in \mathcal{T}$ and applying linearity of expectation gives D . The races are not independent, but linearity does not require independence. $\square$

Proof of Theorem 2. Suppose $\text{Guard}_{\mathcal{M}}(p) = 1$ with witness x' . By Eq. (3) x' was re-executed against p and still failed, so $\neg\phi(p, x')$ holds by Assumption 1. Every stored counterexample was returned by an earlier oracle call and therefore lies in the fault's shipped pool, so p fails a pool input and $p \notin G_{\text{pool}}$; since $G^* \subseteq G_{\text{pool}}$, also $p \notin G^*$. For a particular call, G_k is fixed by that call's draw: if $x' \in X_k$ the call refutes p , so $p \notin G_k$ and blocking p discards nothing that call would have certified. That is automatic whenever X_k exhausts the pool, i.e. $k \geq |\text{pool}|$; otherwise it is an empirical question, which Section 4 settles with an audit rather than an assumption. For the second clause, CEGMEM returns at line 8 of Algorithm 1 only when $\mathcal{O}_k(p_t) = \text{accept}$, i.e. p_t passed every input in that call's X_k . The argument uses only that Eq. (3) re-executes x' ; it is independent of how the search over $\mathcal{M}$ is ordered or restricted, which is why widening the guard's search beyond the λ -matched bucket (Section 5) cannot break soundness. Nothing here yields $\hat{p} \in G^*$: that would require oracle completeness, which we do not assume and instead measure (Section 7.4). $\square$

Proof of Theorem 4. (i) By Assumption 2, $c = 1$, so a stored refutation of type τ places τ in E ; by Eq. (4) the steered proposer has zero mass on E , so no candidate of an eliminated type is drawn. A refuted patch has an eliminated type, hence is never re-proposed. (ii) Consider an oracle call at round t returning a counterexample x_t of type τ_t . By (i), $\tau_t \notin E$ before the call, and after it $\tau_t \in E$, so $|E|$ strictly increases at every refuting call. Since $|E| \leq K$, at most K refuting calls occur, and the run ends on the next call at the latest (either it accepts, or E is exhausted and $\mathcal{G}$'s support is $\{\top\}$, so the next draw is correct). Hence at most $K+1$ oracle calls, and—because (i) makes every proposal reach the oracle—at most $K+1$ proposals. For untyped memory, Eq. (3) still blocks any

candidate a stored counterexample refutes, so the oracle-call bound is unchanged; but the proposer is unconditioned, so the number of proposals is unbounded. With no memory, each round is an independent trial with success probability π , giving $1/\pi$ expected calls and no finite worst case. $\square$

Proof of Theorem 6. (a) No memory: rounds are i.i.d. Bernoulli(π) and every round calls the oracle, so the expected count is $1/\pi$. With memory of either kind, a round calls the oracle iff its type is not yet eliminated (blocked rounds do not call it, by Eq. (3) and Assumption 2). The oracle-calling rounds are therefore exactly the first draw of each distinct type, plus the terminal correct draw: $D+1$ in expectation by Lemma 8. The inequality follows from $q_{\tau} + \pi \geq \pi$ termwise. (b) Redundancy is counted against E^{obs} (Definition 5), which is defined from the episode's refutation history and is therefore available in every arm, including those that store nothing. An unconditioned proposer draws $1/\pi$ proposals in expectation before its first correct one, of which $D+1$ fall in classes not yet in E^{obs} (Lemma 8); the remaining $1/\pi - 1 - D$ are redundant attempts, and this quantity is positive whenever some $q_{\tau} > 0$ by the inequality in (a). It is the expected count for no memory and for untyped memory alike, since neither conditions the proposer. With typed memory, (i) of Theorem 4 says no proposal has a type in $E = E^{\text{obs}}$, so the count is identically 0. Finally, a redundant attempt is paid only if it reaches the oracle: under Eq. (3) every redundant attempt in a memory arm is blocked, so the expected paid count is $1/\pi - 1 - D$ with no memory and 0 with either memory—which is the part of (b) that does not depend on steering. $\square$

Proof of Corollary 3. Both halves use only Theorem 2, so they hold for any sound guard and in particular for untyped memory as well as typed. If blocked rounds are not charged, the episode terminates at the first correct *unblocked* draw, and blocking removes only draws a stored counterexample refutes—draws that are not accepted, by Theorem 2. Deleting non-terminal elements from a sequence cannot postpone the first terminal one, so the success probability within B charged proposals is non-decreasing. It is strictly larger only when deletion moves a terminal draw from beyond B to within it, which requires a blocked round at some $t < B$ in a realization whose first accepting draw would otherwise have fallen outside the budget; a block in a realization that succeeds anyway changes nothing, which is why the strictness condition is about budget position and not about q_{τ} alone. If blocked rounds are charged, the round index of each draw is unchanged and the guard alters no draw's outcome, so the index of the first accepting draw is invariant; under common random numbers the two arms realize the same draw sequence and hence the same index exactly. $\square$

Proof of Proposition 7. Both guards evaluate the existential in Eq. (3) by re-executing stored counterexamples and stopping at the first that still refutes. A flat scan walks memory in insertion order, so it stops at index $\min_{i \in R} i$ if $R \neq \emptyset$ and otherwise consults all m entries. The bucket-first guard walks $\mathcal{B} = \mathcal{M}[\lambda(p)]$ first: if $R \cap \mathcal{B} \neq \emptyset$ it stops within the bucket,

at the position of the first refuting bucket entry; if not, it has consulted $|\mathcal{B}|$ entries for nothing and then walks the remainder, stopping at the first refuting entry there. Neither bound dominates the other pointwise, and when $R = \emptyset$ both consult every entry, so the worst cases coincide and no asymptotic separation holds. The expected saving is therefore the mass R places inside $\mathcal{B}$ weighted by how early it appears there, net of the miss penalty—a distributional property of the type map, which is what Section 7.2 sweeps and finds flat. The last clause is immediate: deduplicating $\mathcal{M}$ by counterexample input and memoizing verdicts within a round leave the set of consultations unchanged while reducing the number that trigger an execution, so a cost measured in seconds can fall while a cost measured in consultations does not. $\square$

B THE POLICY REPLAY

Detail behind Section 7.6: the five validation policies replayed over one NO-MEM candidate stream.

Why the comparison is exact. In NO-MEM the proposer is shown nothing the validator found, so the candidate stream does not depend on which cases ran, in what order, or whether the run stopped early. Five policies therefore replay over byte-identical candidates rather than over separately generated ones. We executed each of the 5,564 distinct NO-MEM candidates against its task’s entire shipped pool—221,304 case executions, 111 h of sandbox time—and replayed the policies against that matrix offline. No cell was dropped and no model was called. The matrix reproduces the counterexample the harness itself found in 8,306 of 8,312 logged oracle rounds; all six exceptions fall on the timeout-edge fault of Section 7.8.

What the replay does not measure. The replay walks all 20 rounds instead of stopping at the first acceptance, which is what keeps the candidate stream policy-independent. Its per-episode levels (19.99 oracle rounds, 4.10 accepts) are therefore not the reported arm’s (9.47 and one), and only ratios within Table 14 are meant to carry over. Repair rate is a property of the stream and is identical by construction; a policy changes only what finding out costs.

How little the axis affords. Among refuted candidates the median share of the shipped pool that refutes is 0.58, so a uniformly random order finds evidence in about 1.7 executions and an ideal order in 1.0: the whole axis the criterion chose is worth about one execution on this corpus. Measured against that floor, MP-AWARE captures 84.6% of the headroom ORACLE- k leaves and CEGMEM-GUARD 83.5%—a gap of 1.1 points, which is the number Section 7.6 reports. A corpus of sparse, hard-to-hit counterexamples would give the axis room to separate mechanisms; this one does not, and we claim nothing about how the loss scales.

The index, measured against its own control. CEGMEM-GUARD differs from DEDUP-GUARD in one respect, consulting the λ bucket before the rest of the store. Oracle calls and

blocked rounds are identical on every one of the 495 cells, so the index moves nothing on the axis the guard is for. It separates only on the execution axes, where the effect is signed and negligible: 46/37/16 tasks on executions to refutation ($p < 10^{-4}$, cell-level medians 1.50 against 1.54) and 45/39/15 on the total, worth 0.25% of it. That is the shape of the typed-versus-untyped result in Table 9, and it points the same way.

C ERRATA IN THE FROZEN ARTIFACTS

Two records in the frozen run are wrong, and we correct them here rather than rewrite a frozen file. `strata.json` reports `n_moved: 0` with a diagonal migration matrix; recomputing from its own per-task $\hat{\pi}$ fields shows 42 of 99 tasks report in a different band than they were selected into (Section 6). And `screening.json` carries a note promising “every candidate in the pool and what became of it ... including everything excluded” above an *empty* list: the candidates screened and then discarded once a band’s quota filled were never recorded, which is why no design weights exist for this corpus. `tools/corpus_addenda.py` recomputes both from the artifacts that are correct, and writes `corpus.json`; neither correction touches any measured outcome.

D SENSITIVITY: THE TIMEOUT-EDGE FAULT

One fault’s candidate patches run at the sandbox’s 30 s wall-clock limit, so its verdicts are timing-dependent (Section 7.8). Table 16 repeats every headline number with that fault removed. The cost and repair results are unmoved—the reduction factors change in the second decimal and the budget curve in the third—and every episode that remains is outcome-identical (294/294 instead of 295/297), which strengthens the pairing claim. One row does move, and we flag it rather than bury it: that single fault contributes 1.8 s of UNTYPED’s 6.66 s guard time, so the guard-second reduction falls from -39% to -18% ($4.85 \rightarrow 3.98$) without it. The sign is unchanged (189/40) but the magnitude is not robust, which is why Section 7.3 reports the clustered interval alongside it.

Table 16: Headline numbers with the timeout-edge fault excluded (98 faults, 490 cells).

	full corpus	excluded
Oracle calls, NO-MEM $\rightarrow$ TYPED	$\times 5.17$	$\times 5.18$
Paid redundancy, NO-MEM $\rightarrow$ TYPED	$\times 56.9$	$\times 56.5$
Repair @ $b=2$, TYPED	0.558	0.557
Repair @ $b=2$, NO-MEM	0.319	0.318
TYPED vs. UNTYPED, oracle	93/59	90/59
Guard sec., UNTYPED $\rightarrow$ GUARD-ONLY	$6.66 \rightarrow 4.04$	$4.85 \rightarrow 3.98$
reduction	-39%	-18%
sign test	191/40	189/40
Episodes outcome-identical	295/297	294/294